Smart Campus Classroom Management System: An IoT-Based Automated Attendance and Environmental Monitoring Platform

Mohamed Hedi Ben Jemaa Ahmed Amine Jallouli Ali Saadaoui
Mediterranean Institute of Technology (SMU)
Tunis, Tunisia
May 2026

***Abstract*—Classroom management in higher education institutions remains largely manual, placing an unnecessary operational burden on faculty and consuming valuable teaching time. At SMU Mediterranean Institute of Technology, professors rely on paper-based or informal digital attendance methods that can consume up to fifteen minutes per lecture, while environmental conditions go unmonitored. This report presents the design, implementation, and evaluation of a Smart Campus Classroom Management System — an Internet of Things (IoT)-based platform that automates attendance tracking, monitors classroom environment in real time, and provides professors with a centralized web dashboard. The system is built around a Raspberry Pi 4B edge hub running a fully containerized stack with no cloud dependency. An ESP32 node publishes temperature, humidity, air quality, and sound data over MQTT every five seconds; a camera performs face recognition at two frames per second using DeepFace/FaceNet; a relay module enables manual and automatic toggling of AC and lighting. The backend is implemented in Python with FastAPI, PostgreSQL, and Redis. A React single-page application delivers live sparklines, attendance tables, controls, and alerts over WebSocket. Attendance is synchronized to Moodle at session end. A locally-hosted phi3-mini model served by Ollama generates natural-language explanations for at-risk students and interprets per-course attendance forecasts, with all data remaining on-premises. The system was delivered across twenty-two iterative development phases within a single academic semester, is fully containerized via Docker Compose, and has been validated on physical hardware.**

***Index Terms*—IoT, Smart Classroom, Face Recognition, MQTT, FastAPI, Edge Computing, Large Language Model, Attendance Automation, Raspberry Pi, Moodle**

Cover Page

MEDITERRANEAN INSTITUTE OF TECHNOLOGY — SMU

ISS — Innovative Systems & Services
Academic Year 2025–2026

Smart Campus Classroom Management System
An IoT-Based Automated Attendance and Environmental Monitoring Platform

Submitted by:
Mohamed Hedi Ben Jemaa
Ahmed Amine Jallouli
Ali Saadaoui
Abdelhamid Ouertani
Iyed Day

Submission Date: May 2026

List of Figures

List of Tables

List of Abbreviations

Abbreviation	Full Term
AC	Air Conditioning
ADC	Analog-to-Digital Converter
API	Application Programming Interface
APScheduler	Advanced Python Scheduler
AQ	Air Quality
BYTEA	Binary Data Type (PostgreSQL)
CoAP	Constrained Application Protocol
CPU	Central Processing Unit
CSS	Cascading Style Sheets
DB	Database
DHT	Digital Humidity and Temperature (sensor family)
DNS	Domain Name System
Docker	Container platform (Docker, Inc.)
ESP32	Espressif Systems ESP32 microcontroller
FPS	Frames Per Second
GPIO	General Purpose Input/Output
GPU	Graphics Processing Unit
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
HTTPS	HTTP Secure
I2C	Inter-Integrated Circuit (serial communication protocol)
IEEE	Institute of Electrical and Electronics Engineers
IoT	Internet of Things
JSONB	JSON Binary (PostgreSQL data type)
JWT	JSON Web Token
LAN	Local Area Network
LCD	Liquid Crystal Display
LLM	Large Language Model
LMS	Learning Management System
MCU	Microcontroller Unit
MQTT	Message Queuing Telemetry Transport
nginx	HTTP server and reverse proxy
NX	Not eXists (Redis SET NX flag)
OOM	Out of Memory
ORM	Object-Relational Mapper
PhD	Doctor of Philosophy
PPM	Parts Per Million
QoS	Quality of Service
RAM	Random Access Memory
RBAC	Role-Based Access Control
Redis	Remote Dictionary Server
REST	Representational State Transfer
RFID	Radio Frequency Identification
RPi	Raspberry Pi
SBC	Single-Board Computer
SMU	Schiller Mediterranean University (Mediterranean Institute of Technology (SMU))
SPA	Single-Page Application
SQL	Structured Query Language
SSD	Solid-State Drive
TCP	Transmission Control Protocol
TF	TensorFlow
TLS	Transport Layer Security
TTL	Time To Live
UI	User Interface
UUID	Universally Unique Identifier
VOC	Volatile Organic Compound
WS	WebSocket
WSS	WebSocket Secure

I. Introduction

A. Context and Motivation

Higher education institutions are increasingly expected to leverage digital infrastructure not only for course delivery and academic content management, but also for the physical management of learning spaces. At the Mediterranean Institute of Technology (SMU), a growing institution serving approximately 2,000 students across more than 30 classrooms, the

gap between digital academic tools and physical classroom operations remains significant. Professors rely on platforms such as Moodle for course content distribution and grade management, yet the operational tasks associated with each classroom session — recording student attendance, regulating environmental conditions, and responding to physical classroom anomalies — are still performed entirely by hand.

The academic literature on smart campus infrastructure has demonstrated that the convergence of Internet of Things (IoT) sensing, edge computing, and lightweight machine learning presents a viable path toward automating these operational tasks without the latency, cost, or privacy exposure introduced by cloud-dependent architectures [1], [2]. Environmental quality in learning spaces has also been shown to have a measurable effect on student cognitive performance; elevated CO_2 concentrations and sub-optimal temperatures are associated with reduced attention and academic outcomes [3]. Despite this evidence, the typical university classroom continues to rely on manual intervention for environmental regulation.

At Mediterranean Institute of Technology (SMU), with an estimated 80 to 100 lecture sessions delivered daily across its classrooms, the cumulative inefficiency of manual processes is substantial. A conservative estimate of five to fifteen minutes spent per session on attendance alone translates to several hours of lost instructional time each day at the institutional level. The absence of any real-time feedback loop between classroom conditions and the faculty means that environmental degradation — rising temperatures, deteriorating air quality, or malfunctioning equipment — may go unaddressed for the duration of an entire lecture.

These conditions motivate the development of a dedicated, on-premises smart classroom management system tailored to the operational context of Mediterranean Institute of Technology (SMU).

B. Problem Statement

Three distinct but interrelated operational problems have been identified within the current classroom management practice at Mediterranean Institute of Technology (SMU):

Problem 1 — Attendance Inefficiency. Attendance recording is performed manually at the start of each session, either by verbal roll call or by circulating paper sheets. This process consumes between five and fifteen minutes of teaching time per session and introduces opportunities for error, proxy attendance, and inconsistent record-keeping. The resulting attendance data is not automatically integrated into the institution's Moodle LMS, requiring additional manual data entry by faculty.

Problem 2 — Unmonitored Environmental Conditions. Classrooms at Mediterranean Institute of Technology (SMU) are not equipped with any sensor infrastructure for real-time monitoring of temperature, humidity, air quality, or occupancy. AC and lighting systems are controlled manually, with no mechanism for automated response to threshold conditions. The absence of environmental data prevents any analysis of the relationship between classroom comfort and attendance or academic performance.

Problem 3 — No Early Warning System for At-Risk Students. The institution currently has no automated mechanism to identify students whose attendance patterns place them at academic risk. Advising departments rely on periodic manual reviews of attendance registers, which are inherently delayed and may fail to surface patterns early enough for effective intervention. No tool exists to provide professors or advisors with actionable, per-student attendance analytics in real time.

Collectively, these problems represent a significant operational gap that is addressable through the deliberate application of IoT sensing, edge computing, computer vision, and local AI inference.

C. Project Objectives

1) General Objective: The general objective of this project is to design, implement, and validate an IoT-based Smart Campus Classroom Management System for Mediterranean Institute of Technology (SMU) that automates student attendance tracking, monitors and regulates classroom environmental conditions in real time, and provides professors and administrators with a centralized data-driven dashboard — all operating entirely on-premises without any dependency on external cloud services.

2) Specific Objectives: The following six specific objectives were defined at the outset of the project and guided its development throughout:

1) **Reduce manual attendance overhead** — Automate student identification and attendance recording through camera-based face recognition, eliminating the need for roll calls or paper registers and targeting a reduction of up to 15 minutes of session time currently spent on attendance.
2) **Monitor classroom environmental conditions** — Continuously acquire and display real-time data for temperature, humidity, air quality (CO_2/VOC proxy), and sound presence levels using calibrated sensor hardware deployed within the classroom.
3) **Enable automated and manual environmental control** — Implement relay-based actuation for AC and lighting systems, supporting both automatic threshold-driven control and manual override by the professor via the dashboard.
4) **Provide a centralized professor-facing dashboard** — Deliver a web-based interface accessible from any device on the campus LAN that consolidates live sensor data, attendance records, session history, alerts, and classroom control functions in a single unified view.
5) **Generate real-time alerts for anomalous conditions** — Detect and notify professors of environmental threshold breaches (excessive temperature, poor air quality), device connectivity failures, and unusual attendance patterns, delivering alerts both on the dashboard and on the classroom LCD display.
6) **Synchronize attendance data with Moodle** — Automatically push finalized session attendance records to the institution's Moodle LMS at the close of each session,

with a Redis-backed retry mechanism to handle temporary LMS unavailability.

D. Scope and Limitations

The scope of this project is defined as follows. The system is designed and validated for deployment in a single classroom at Mediterranean Institute of Technology (SMU), identified as `room1` in the system configuration. The software architecture supports multi-room extension through configuration changes, but multi-room operation has not been tested within this project. All services — including the MQTT broker, backend, database, cache, AI inference engine, and web frontend — run on a single Raspberry Pi 4B host, either natively or within Docker containers, and are accessible only to devices connected to the campus local area network.

The following limitations are acknowledged:

- The MQ-135 air quality sensor provides a relative CO_2 and VOC proxy reading and has not been calibrated against a certified reference instrument. Readings are suitable for threshold-based alerting but should not be treated as precise absolute measurements.
- Face recognition accuracy is subject to variation with ambient lighting conditions, camera angle, and occlusion (e.g., face masks, glasses). The system operates at two frames per second on the Raspberry Pi 4B, which constrains throughput relative to higher-specification hardware.
- The local LLM (phi3-mini running via Ollama) inference is performed on the Raspberry Pi CPU, resulting in response times of 10 to 30 seconds per student for the at-risk explanation pipeline. This is mitigated by asynchronous background processing and progressive frontend polling.
- The current deployment does not implement HTTPS or WSS, which is acceptable for a closed university intranet but would require nginx SSL termination before any public-facing deployment.
- The project was completed within a single academic semester, and certain features identified during development — including a mobile application, multi-room support, and QR-code self-enrollment — are deferred to future work.

E. Report Structure

The remainder of this report is organized as follows.

Chapter II reviews relevant literature on smart classroom systems, face recognition-based attendance, IoT communication protocols, edge AI inference, and LMS integration, and positions the present work relative to existing solutions.

Chapter III presents the requirements analysis, including stakeholder identification, functional and non-functional requirements, use case modeling, and project constraints.

Chapter IV describes the full system architecture, covering each subsystem — IoT firmware, edge computing, MQTT communication, backend services, real-time WebSocket layer, data layer, frontend dashboard, and external integrations — along with the key architectural decisions that shaped the design.

Chapter V details the implementation process, including hardware assembly, firmware development, backend and frontend development, containerized deployment, and the iterative development methodology supported by Notion-based task management.

Chapter VI covers the AI and analytics layer in depth, including the at-risk student detection pipeline, the attendance forecasting pipeline, the SQL-driven insights engine, and the rationale for the hybrid deterministic-plus-LLM classification approach.

Chapter VII addresses security and privacy considerations, covering authentication, authorization, biometric data handling, transport security, and operational hardening.

Chapter VIII presents the testing and validation strategy and results, including sensor accuracy, face recognition performance, API testing, real-time latency, AI pipeline performance, and resilience testing.

Chapter IX discusses the overall results of the project relative to the stated objectives, documents lessons learned, and acknowledges remaining limitations.

Chapter X concludes the report and outlines directions for future work.

II. Literature Review & Related Work

A. Smart Classroom Systems — State of the Art

The concept of the smart classroom has evolved considerably over the past decade, driven by the proliferation of low-cost IoT hardware, the maturation of edge computing platforms, and growing institutional interest in data-driven education management. Early smart classroom implementations focused primarily on interactive whiteboards and audience response systems, with limited integration of physical sensing infrastructure [4]. More recent systems have expanded this scope to include environmental monitoring, automated attendance, and adaptive lighting or HVAC control.

Radio Frequency Identification (RFID)-based attendance systems represent one of the earliest and most widely deployed approaches to automated attendance in educational settings. In these systems, each student carries an RFID card that is scanned at a reader installed at the classroom entrance [5]. While reliable and low-cost, RFID-based systems require students to actively present their cards and are vulnerable to proxy attendance — a student scanning the card of an absent peer. They also provide no environmental sensing capability and no integration with institutional analytics pipelines.

QR-code-based attendance systems address the proxy attendance problem only partially. Students scan a session-specific QR code displayed by the professor, which is time-limited to prevent sharing [6]. These systems are easy to deploy on existing mobile devices but remain dependent on student compliance and cannot passively detect occupancy or environmental conditions.

Camera-based and computer vision attendance systems represent the current frontier of automated attendance research.

By leveraging face recognition algorithms, such systems can passively identify students as they enter a classroom without requiring any active participation from either the student or the professor [7]. The primary challenges reported in the literature are computational cost on resource-constrained hardware, sensitivity to lighting conditions and camera placement, and the need for a reliable face enrollment database.

The system developed in this project belongs to this third generation of smart classroom platforms. It combines passive face recognition attendance with real-time environmental sensing, relay-based physical control, LMS integration, and an AI-powered analytics layer — a combination that, as shown in Section II-F, is not present in any single comparable published system.

B. Face Recognition in Attendance Systems

Face recognition for attendance automation has been an active area of research since the early 2010s, with significant advances driven by deep learning. The dominant paradigm in current systems is the use of deep convolutional neural networks to produce compact face embeddings — fixed-length numerical vectors that encode facial identity — which are then compared against a database of enrolled embeddings using a distance metric.

The FaceNet architecture, introduced by Schroff et al. [8], demonstrated that a 128-dimensional embedding trained with a triplet loss function could achieve near-human face verification accuracy on standard benchmarks. The DeepFace library [9], used in this project, provides a unified Python interface to multiple pre-trained face recognition models including FaceNet, and supports cosine distance as the similarity metric. A cosine distance below a configurable threshold (set to 0.6 in this system) indicates a match between a detected face and an enrolled student.

On resource-constrained edge hardware such as the Raspberry Pi 4B, deep learning inference is computationally expensive. Studies have reported frame processing times of 300 to 800 milliseconds per frame for FaceNet-based recognition on ARM Cortex-A class processors [10], which constrains practical throughput to approximately 2 frames per second — the rate adopted in this system. This is sufficient for the use case of identifying students as they enter a classroom, where the subject is typically within camera view for several seconds.

A key consideration in any biometric attendance system is the handling of enrolled face data. This project stores only the averaged 128-dimensional float32 embedding per student, not the source images, and all processing occurs locally on the Raspberry Pi. No biometric data is transmitted to any external service, which is consistent with best practices for biometric data minimization in educational environments [11].

C. IoT Protocols for Educational Environments

The selection of a communication protocol between IoT sensor nodes and a backend processing system involves tradeoffs between bandwidth efficiency, latency, reliability, and implementation complexity. The three protocols most commonly evaluated in IoT educational deployments are MQTT, HTTP, and CoAP.

HTTP is the most familiar protocol and is natively supported by virtually all platforms, but its request-response model introduces significant overhead for high-frequency sensor publishing. Each HTTP POST from a sensor node requires a full TCP handshake and header exchange, making it inefficient for payloads as small as a temperature reading published every five seconds [12].

CoAP (Constrained Application Protocol) was designed specifically for constrained IoT devices and operates over UDP, offering lower overhead than HTTP. However, CoAP's UDP transport requires additional mechanisms for reliability and is less well-supported by backend frameworks and broker infrastructure than MQTT [12].

MQTT (Message Queuing Telemetry Transport) is a publish-subscribe protocol designed for low-bandwidth, high-latency, or unreliable networks. It operates over a persistent TCP connection to a broker, which decouples publishers (sensor nodes) from subscribers (backend services). The protocol's minimal fixed header (as small as 2 bytes) makes it highly efficient for frequent small-payload sensor publishing [13]. MQTT supports three Quality of Service levels: QoS 0 (at most once, fire-and-forget), QoS 1 (at least once), and QoS 2 (exactly once). This project uses QoS 0 for sensor telemetry, which is appropriate given the high publication frequency (every 5 seconds) — occasional message loss is acceptable because a fresh reading will arrive within the next interval.

The Eclipse Mosquitto broker, deployed as a Docker container on the Raspberry Pi, serves as the MQTT message bus between the ESP32 sensor node and the FastAPI backend. This architecture is consistent with published recommendations for campus IoT deployments [14].

D. Local LLM Inference at the Edge

The integration of large language models (LLMs) into operational systems has historically been dominated by cloud API approaches, where inference is delegated to remote servers operated by providers such as OpenAI or Anthropic. While cloud LLM APIs offer high model quality and zero local compute requirements, they introduce latency, recurring API costs, and — critically for educational systems handling student data — privacy exposure through the transmission of personally identifiable information to third-party services [15].

The emergence of quantized, small-parameter LLMs designed for CPU inference has created a viable alternative for edge deployment. Models in the 3–7 billion parameter range, when quantized to 4-bit precision, can be loaded and served on consumer-grade hardware including single-board computers. The phi3-mini model [16], developed by Microsoft Research, achieves competitive performance on reasoning and text generation benchmarks while requiring as little as 2.3 GB of memory in its quantized form, making it suitable for deployment on the Raspberry Pi 4B (4 GB RAM).

Ollama [17] provides a runtime and HTTP API for serving quantized LLMs locally, with a model management interface

analogous to Docker. In this project, Ollama is deployed as a Docker container on the Raspberry Pi and exposes a REST API consumed by the FastAPI backend. All student data used in LLM prompts — attendance rates, missed session statistics, peer comparisons — remains within the campus local area network at all times.

The use of a local LLM in this system is deliberately constrained to natural language generation tasks (prose summaries and forecast interpretations) where structured outputs are not required. Structured classification and scoring tasks are handled by deterministic algorithms, as discussed in Chapter VI.

E. LMS Integration

Learning Management Systems are the primary digital interface between faculty and students in most higher education institutions. Moodle, the open-source LMS used at Mediterranean Institute of Technology (SMU), provides a comprehensive REST API that supports programmatic read and write operations on course data, user records, grades, and attendance [18].

Prior work on automated Moodle attendance integration has typically followed one of two approaches: direct API push from a standalone attendance application [19], or integration via Moodle plugins such as the Attendance activity module. The direct API approach, adopted in this project, offers greater flexibility and does not require modification of the Moodle installation. Attendance records are pushed to Moodle at the close of each session via an authenticated REST call using a service token with scoped permissions.

A known challenge with LMS integration in IoT systems is the handling of network unavailability between the IoT backend and the LMS. This project addresses this through a Redis-backed retry queue: if a Moodle sync call fails, the session identifier is enqueued in Redis and retried by a scheduled background job every ten minutes, for up to ten attempts.

F. Summary and Positioning of This Work

Table I presents a comparative analysis of four representative smart classroom systems from the literature alongside the system developed in this project. The comparison covers the key feature dimensions relevant to the problem statement defined in Chapter I.

As shown in Table I, no single comparable system combines passive face recognition attendance, full environmental monitoring with relay actuation, automatic LMS synchronization, and an AI-powered analytics layer within a cloud-free, edge-deployed architecture. The system presented in this report addresses all five feature gaps identified in the related work and introduces a locally-hosted LLM inference capability not found in any of the surveyed systems.

III. Requirements Analysis

A. Stakeholder Analysis

The Smart Campus Classroom Management System serves a set of stakeholders with distinct but overlapping needs. Four stakeholder groups were identified through analysis of the institutional context at Mediterranean Institute of Technology (SMU).

Professors (Primary Users). Professors are the direct operators of the system during classroom sessions. Their primary needs are the elimination of manual attendance overhead, real-time visibility into classroom environmental conditions, the ability to override automated AC and lighting decisions, and access to session history and attendance analytics. The system must present information to professors with minimal interaction cost — a professor managing a live lecture cannot be expected to navigate complex interfaces.

Students (Indirect Beneficiaries). Students do not interact with the system directly during sessions. They benefit indirectly through more accurate and timely attendance records, improved classroom comfort resulting from automated environmental regulation, and the potential for earlier academic intervention enabled by the at-risk detection pipeline. Students interact with the system during the enrollment phase, providing face images for recognition registration.

SMU Administration (Secondary Users). Administrative staff and department heads require aggregated, institution-level visibility into classroom utilization, attendance trends, and environmental data. The system's role-based access control model supports an admin role with access to all courses, all students, and all pipeline recomputation functions, providing a foundation for institutional reporting and resource optimization.

Advising and Counseling Department (Secondary Users). The advising function at Mediterranean Institute of Technology (SMU) benefits from the at-risk student detection capability. Automated identification of students whose attendance falls below 70%, combined with natural-language explanations generated by the local LLM, enables earlier and better-informed outreach compared to periodic manual register reviews.

B. Functional Requirements

The following functional requirements were derived from the stakeholder analysis and the six specific project objectives defined in Chapter I. Requirements are identified by unique IDs and grouped by subsystem. Table II lists all functional requirements.

C. Non-Functional Requirements

Table III lists the non-functional requirements of the system.

D. Use Case Diagram

Fig. 1 presents the UML use case diagram for the Smart Campus Classroom Management System. Two primary actors are identified: the **Professor**, who manages live classroom sessions and accesses operational data, and the **Administrator**, who extends the Professor's capabilities with system-level management functions. A **Student** actor is included for the enrollment use case, which involves student participation during face image registration. The **ESP32 Node** and **Moodle LMS** are represented as external system actors interacting with the system boundary.

TABLE I
RELATED WORK COMPARISON.

Feature	Rahman et al. [20]	Singh et al. [21]	Won & Park [22]	Al-Turjman et al. [23]	This Work
Attendance method	RFID card scan	QR code (mobile)	Face recognition	RFID + fingerprint	Face recognition (passive)
Environmental monitoring	None	None	Temperature only	Temperature, humidity	Temp, humidity, AQ, sound
AC / lighting control	None	None	None	None	Yes — relay + MQTT
Real-time dashboard	Basic web page	Mobile app	Web dashboard	None	React SPA + WebSocket
LMS integration	Manual export	None	Moodle (manual)	None	Moodle REST API (automatic)
AI / analytics layer	None	None	None	Anomaly detection	At-risk LLM + forecasting
Cloud dependency	Yes	Yes	Yes	Yes	None — fully on-premises
Edge deployment	No	No	Partial	No	Full (RPi 4B + Docker)
Open source / reproducible	Partial	No	No	No	Yes — Docker Compose

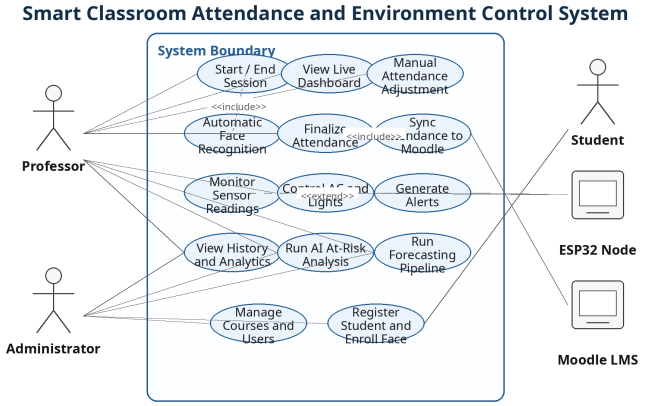

Fig. 1. UML use case diagram.

E. Constraints

The following constraints bounded the design and implementation decisions throughout the project:

Hardware availability and cost. The bill of materials required for this project, detailed in Chapter V, represents a significant financial commitment for a student team. The Raspberry Pi 4B (4GB), camera module, ESP32 development board, and relay module were acquired specifically for this project. Component procurement timelines and the high cost of electronic parts in the Tunisian market introduced scheduling risk, particularly in the early phases of the project.

Computational constraints of the Raspberry Pi 4B. All services — including face recognition inference (DeepFace/FaceNet) and local LLM inference (phi3-mini via Ollama) — run on the Raspberry Pi 4B's ARM Cortex-A72 CPU with 4 GB of RAM. Face recognition is constrained to 2 frames per second to remain within real-time processing bounds. LLM inference takes 10 to 30 seconds per student, mitigated by asynchronous background processing. These constraints would be substantially reduced by upgrading to a Raspberry Pi 5 or a host with a dedicated GPU.

Single-semester delivery. The project was required to be fully designed, implemented, and validated within one academic semester, with continuous progress milestones. This constraint necessitated an iterative, phase-based development approach and required that certain planned features — including multi-room support, a mobile application, and IR illuminator hardware — be deferred to future work.

Single-room scope. The system was designed and validated for a single classroom. While the architecture supports multi-room extension via configuration, this capability was not tested within the project timeline.

IV. SYSTEM ARCHITECTURE

A. Architecture Overview

The Smart Campus Classroom Management System is structured as a **hybrid modular monolith with event-driven messaging**. The backend is implemented as a single FastAPI process rather than a distributed microservices architecture. This decision reduces operational complexity, eliminates inter-service network overhead, and is appropriate for the single-host deployment target (Raspberry Pi 4B). Internally, the backend is event-driven through three decoupled asynchronous channels — one for sensor events, one for attendance events, and one for alert events — each backed by an `asyncio.Queue` instance that decouples producers from WebSocket consumers.

Four architectural paradigms govern the system's design:

Publish/Subscribe via MQTT (QoS 0). The ESP32 sensor node publishes readings to the Mosquitto broker, which delivers them to the backend's persistent MQTT subscriber loop. Relay commands travel in the reverse direction. This paradigm decouples the physical IoT layer from the application layer and provides a well-defined, topic-structured message bus.

Push-based real-time delivery via WebSocket. Rather than polling, the frontend receives live sensor readings, attendance events, and alerts via a persistent WebSocket connection to `WS`

TABLE II
FUNCTIONAL REQUIREMENTS.

ID	Subsystem	Requirement
FR-01	Attendance	The system shall automatically detect and record student presence using camera-based face recognition during active sessions.
FR-02	Attendance	The system shall assign a default status of "absent" to enrolled students not detected during a session upon session closure.
FR-03	Attendance	The system shall allow professors to manually adjust individual attendance records (present, absent, late, excused) after automatic detection.
FR-04	Attendance	The system shall enforce a 30-second per-student cooldown to prevent duplicate attendance records within a single session.
FR-05	Attendance	The system shall synchronize finalized attendance records to the Moodle LMS upon session end, with automatic retry on failure.
FR-06	Environment	The system shall continuously acquire and publish temperature and humidity readings from the DHT21 sensor at intervals not exceeding 5 seconds.
FR-07	Environment	The system shall continuously acquire and publish air quality (CO_2/VOC proxy) readings from the MQ-135 sensor at intervals not exceeding 5 seconds.
FR-08	Environment	The system shall continuously acquire and publish sound presence readings from the ACP014 sensor at intervals not exceeding 5 seconds.
FR-09	Environment	The system shall display current sensor readings on the classroom LCD display at all times.
FR-10	Control	The system shall actuate the AC relay in response to commands issued via the dashboard or triggered automatically when temperature thresholds are exceeded in auto mode.
FR-11	Control	The system shall actuate the lighting relay in response to commands issued via the dashboard.
FR-12	Control	The ESP32 node shall enforce AC temperature thresholds locally (on-device) to maintain control if the MQTT connection to the backend is unavailable.
FR-13	Dashboard	The system shall provide a web-based dashboard displaying live sensor readings, updated in real time via WebSocket.
FR-14	Dashboard	The system shall display current session status, attendance list, and relay states on the dashboard.
FR-15	Dashboard	The system shall provide professors with the ability to start and end classroom sessions from the dashboard.
FR-16	Dashboard	The system shall display session history with per-session attendance and environmental summaries.
FR-17	Alerts	The system shall generate an alert when the classroom temperature exceeds the configured high threshold (default: 28°C).
FR-18	Alerts	The system shall generate an alert when the air quality reading exceeds the configured threshold (default: 500 ppm).
FR-19	Alerts	The system shall generate an alert when the ESP32 device heartbeat is not received within 60 seconds (device offline).
FR-20	Alerts	The system shall suppress duplicate alerts of the same type for the same room while a prior unacknowledged alert of that type exists.
FR-21	Enrollment	The system shall allow administrators to register new students and upload face images for recognition enrollment.
FR-22	Enrollment	The system shall compute and store a 128-dimensional face embedding for each enrolled student, derived from up to five uploaded images.
FR-23	AI — At-Risk	The system shall identify students whose overall attendance rate falls below a configurable threshold (default: 70%) and generate a natural-language explanation for each such student.
FR-24	AI — At-Risk	The system shall display per-course attendance statistics alongside LLM-generated explanations on the at-risk dashboard page.
FR-25	AI — Forecasting	The system shall compute an attendance trend classification (steady_decline, accelerating_decline, stable, recovering) for each course using a deterministic algorithm.
FR-26	AI — Forecasting	The system shall generate a natural-language interpretation and projected next-session attendance rate for each course using the locally-hosted LLM.
FR-27	Auth	The system shall authenticate professor and admin users via JWT tokens and enforce role-based access control on all API endpoints.

`/ws/classroom/{room_id}`. On connection, a snapshot of current Redis state is delivered immediately to populate the dashboard without waiting for the next MQTT message.

Queue-based fan-out via `asyncio.Queue`. Three background drain tasks run continuously in the FastAPI event loop, consuming from their respective queues and broadcasting to all connected WebSocket clients for the given room. This decoupling means that the MQTT handler, the face recognition loop, and the alert engine never block on WebSocket delivery.

On-demand async AI pipelines. The at-risk and forecasting LLM pipelines are not scheduled by a cron job but are triggered on-demand when a professor navigates to the relevant dashboard page. Redis locks with TTL-based natural cooldowns prevent concurrent or excessively frequent pipeline runs.

Fig. 2 presents the full layered system architecture diagram.

B. IoT Layer — ESP32 Firmware

The ESP32 development board (ESP-WROOM-32 with CP2102 USB-UART) serves as the classroom sensor node. It runs a continuous acquisition loop implemented in C++ using the Arduino IDE framework and communicates with the Raspberry Pi backend exclusively over WiFi via MQTT.

Sensor acquisition. Every five seconds, the firmware reads temperature and relative humidity from the DHT21 (AM2301) sensor using the DHT library, reads an analog voltage from the MQ-135 air quality sensor via the ESP32's ADC (converted to a comparative PPM proxy value), and reads a digital HIGH/LOW signal from the ACP014

TABLE III
NON-FUNCTIONAL REQUIREMENTS.

ID	Category	Requirement
NFR-01	Performance	Sensor readings shall be delivered to the dashboard within 5 seconds of acquisition end-to-end (ESP32 → MQTT → backend → WebSocket → browser).
NFR-02	Performance	The WebSocket snapshot on dashboard connect shall be served from Redis within 100 ms, without querying PostgreSQL.
NFR-03	Performance	The alert engine shall evaluate sensor thresholds and publish alerts within a 30-second polling cycle.
NFR-04	Availability	All system services shall operate entirely on the campus local area network with no dependency on external internet connectivity.
NFR-05	Availability	The frontend dashboard shall remain functional in demo mode (client-side mock data) when the backend is fully offline.
NFR-06	Availability	The MQTT bridge shall reconnect to the Mosquitto broker with exponential backoff (3s to 30s maximum) following a connection failure.
NFR-07	Security	All professor and admin passwords shall be stored as bcrypt hashes. No plaintext credentials shall be persisted anywhere in the system.
NFR-08	Security	All student biometric data (face embeddings) shall be stored exclusively on-premises. No biometric data shall be transmitted to any external service.
NFR-09	Security	Role-based access control shall ensure that professors can only access data for their own courses and students.
NFR-10	Privacy	LLM inference shall be performed locally via Ollama. No student data shall be included in requests to any external API.
NFR-11	Scalability	The software architecture shall support extension to multiple rooms through configuration changes (ROOM_ID), without requiring code modifications.
NFR-12	Maintainability	Database schema migrations shall be applied automatically at backend startup via Alembic, requiring no manual migration steps.
NFR-13	Deployability	The full system stack shall be deployable on a Raspberry Pi 4B (4GB RAM) using a single `docker compose up -d` command.
NFR-14	Resilience	The system shall provide a mock sensor mode (`MOCK_MODE=true`) that replaces ESP32 hardware with synthetic sensor data, enabling full software validation without physical hardware.

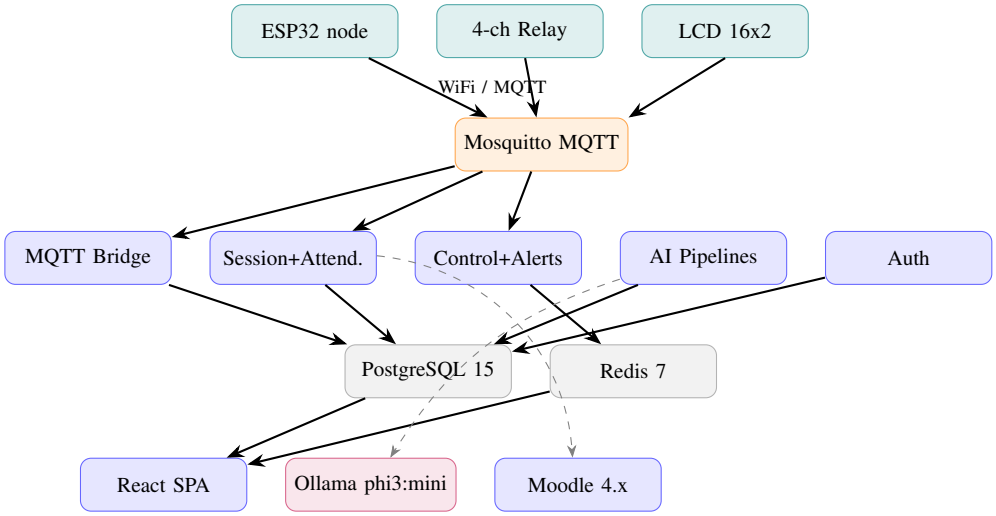

Fig. 2. Full system layered architecture diagram.

sound detection module. Each reading is serialized as a compact JSON payload {"value": <float>, "unit": "<string>", "ts": <unix_ms>} and published to the corresponding MQTT topic.

Heartbeat. Every 30 seconds, the firmware publishes a status heartbeat {"online": true, "ts": <unix_ms>} to classroom/{room_id}/status. The backend's MQTT bridge renews a Redis TTL key on receipt of this message. If no heartbeat arrives within 60 seconds, the key expires and the alert engine generates a device_offline alert.

Relay actuation. The firmware subscribes to classroom/{room_id}/relay/ac and classroom/{room_id}/relay/lighting. On receipt of {"action": "on"}, {"action": "off"}, or {"action": "auto"}, the corresponding GPIO pin driving the 4-channel opto-isolated relay module is set HIGH or LOW. A Logic Level Converter (3.3V ↔ 5V) is used between the ESP32 GPIO outputs and the relay module inputs, since the relay module requires 5V logic signals while the ESP32 operates at 3.3V.

On-device auto-control. When acAutoMode is set to true in config.h, the ESP32 evaluates temperature against

TABLE IV
SOFTWARE AND PLATFORM STACK SUMMARY.

Layer	Technology	Version
Firmware	Arduino IDE / C++	—
MCU libraries	PubSubClient, DHT, LiquidCrystal_I2C, ArduinoJson	—
Backend language	Python	3.11
Backend framework	FastAPI + Uvicorn	latest
ORM	SQLAlchemy (async)	2.0
Migrations	Alembic	latest
Relational DB	PostgreSQL	15
Cache	Redis	7
MQTT client	aiomqtt	2.x
HTTP client	httpx (async)	latest
Face recognition	DeepFace / FaceNet	latest
Computer vision	opencv-python-headless	latest
Scheduler	APScheduler	latest
Local LLM runtime	Ollama	latest
Local LLM model	phi3:mini	—
Frontend framework	React + Vite	18 / 5
Styling	TailwindCSS v3 + CSS custom properties	3
Charts	Recharts	latest
HTTP reverse proxy	nginx	latest
Containerization	Docker Compose	latest
MQTT broker	Eclipse Mosquitto	2

TABLE V
SUBSYSTEM RESPONSIBILITIES.

Subsystem	Technology	Primary Responsibility
IoT Layer	ESP32 (WROOM-32)	Sensor acquisition, relay actuation, LCD display, on-device auto-control
Edge Runtime	Raspberry Pi 4B	Host for all containerized services; camera face recognition
MQTT Broker	Eclipse Mosquitto 2	Message bus between ESP32 and backend
Backend	FastAPI (Python 3.11)	REST API, WebSocket, MQTT bridge, alert engine, AI pipeline orchestration
Real-time Layer	asyncio.Queue + WebSocket	Fan-out of sensor, attendance, and alert events to browser clients
AI / Insights	Ollama phi3:mini + SQL	At-risk explanation, attendance forecasting, SQL analytics
Frontend	React 18 + Vite	Professor-facing SPA dashboard; demo mode when backend offline
Data Layer	PostgreSQL 15 + Redis 7	Durable persistent records + low-latency live state cache
LMS Integration	Moodle 4.x REST API	Attendance synchronization at session end
LLM Runtime	Ollama (local Docker)	On-premises phi3-mini inference; no cloud API

the configured thresholds (28°C on, 22°C off) and actuates the AC relay independently, without waiting for a command from the backend. This is a deliberate redundancy: the backend's alert engine also sends relay commands, but the device can maintain safe thermal conditions even if the WiFi connection or MQTT broker is temporarily unavailable.

LCD display. The firmware drives a 16×2 LCD display via I2C (address 0x27) using the `LiquidCrystal_I2C` library. The display shows the four most recent sensor readings (temperature, humidity, air quality, sound status) in a two-row scrolling format, providing local feedback without requiring the professor to open the dashboard.

C. Edge Layer — Raspberry Pi 4B

The Raspberry Pi 4B (4GB RAM) serves as the single edge computing hub for the entire system. In production, all services run as Docker containers on the RPi. In development on a standard laptop, the same `docker compose up -d` command brings up an identical stack, with the face recognition service replaced by a stub and the ESP32 hardware replaced by a software mock sensor publisher.

Camera and face recognition subsystem. The Raspberry Pi Camera Module v2 (8MP) is connected to the RPi's dedicated CSI camera port. When a session is active, an `asyncio.Task` (`recognition_loop.py`) captures frames at 2 fps using OpenCV's `VideoCapture(0)`. Each frame is passed to `face_recognition_service.py`, which invokes DeepFace with the FaceNet backend to detect faces and compute 128-dimensional embedding vectors. Each embedding is compared against all stored student encodings using cosine distance; a match is declared when the distance falls below the configured threshold (default: 0.6). The first confirmed match for a given student within a session triggers an attendance record insertion, with a 30-second per-student cooldown to prevent duplicates caused by the student remaining in frame.

When `FACE_RECOGNITION_ENABLED=false` (the default in Docker, since TensorFlow is not included in the Docker image to avoid the ∼600MB pull and OOM risk on development laptops), the `_stub_recognition_loop` emits one synthetic attendance event every 45 seconds for a randomly selected enrolled student. This allows full end-to-end testing of the attendance pipeline, WebSocket fan-out, and dashboard update cycle without physical hardware.

Mock sensor mode. When `MOCK_MODE=true`, the `mock_sensor.py` service uses APScheduler to publish synthetic MQTT messages every 5 seconds. Temperature follows a sine wave between 22°C and 32°C with added noise; humidity varies between 45% and 70%; air quality occasionally spikes above the 500 PPM alert threshold; sound presence is randomly toggled at 70%/30%. The Mosquitto broker receives these messages exactly as it would from an ESP32 — the MQTT bridge is entirely unaware of the source.

Frontend demo mode. Even when the backend is completely offline, the React frontend activates a client-side mock data generator if no WebSocket data arrives within 8 seconds of connecting. This ensures the dashboard remains navigable and

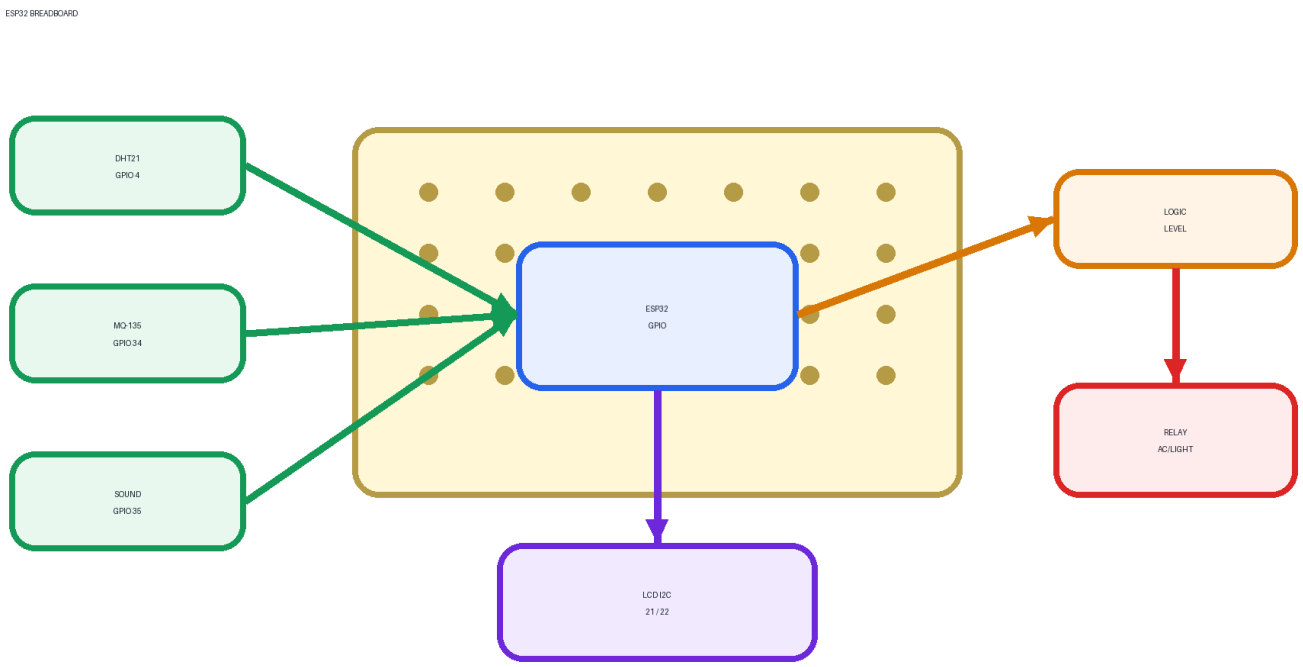

Fig. 3. ESP32 breadboard with sensors and relay module.

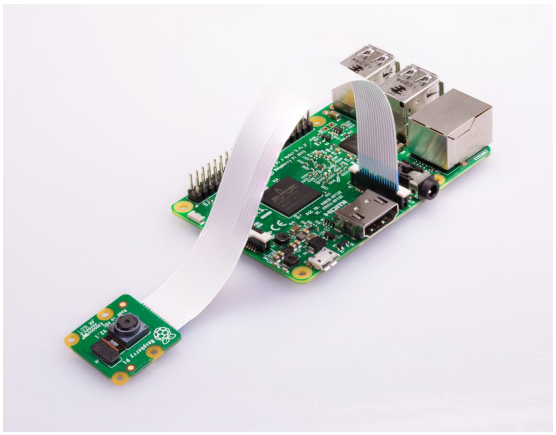

Fig. 4. Raspberry Pi 4B with camera module.

visually representative during presentations or demonstrations without a live hardware stack.

D. Communication Layer — MQTT

The Eclipse Mosquitto broker (version 2, deployed as a Docker container) serves as the message bus between the ESP32 sensor node and the FastAPI backend. All MQTT communication occurs over TCP port 1883 on the campus LAN.

Topic architecture. All topics follow the namespace `classroom/{room_id}/...`, where `room_id` defaults to `room1` and is configurable via environment variable. This namespace design provides a clear scoping mechanism for future multi-room extension. Fig. 5 illustrates the full topic tree.

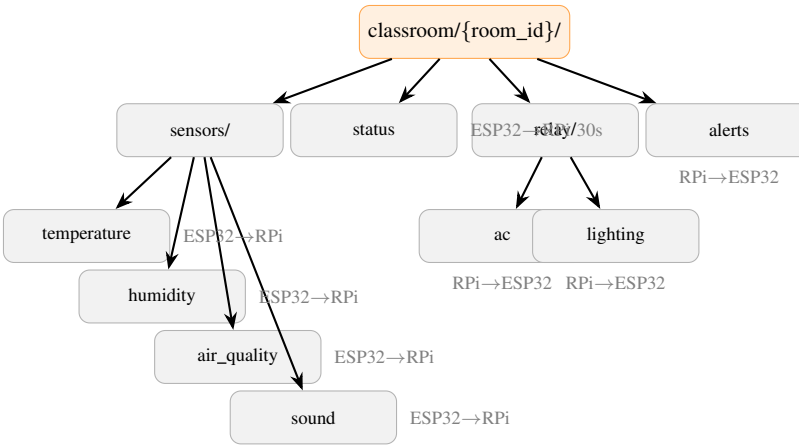

Fig. 5. MQTT topic tree.

Subscription strategy. The backend subscribes to the wildcard patterns `classroom/+/sensors/#` and `classroom/+/status`, where `+` matches any single-level segment (room ID) and `#` matches any multi-level suffix. This allows the backend to handle messages from any room without reconfiguring subscriptions. The ESP32 subscribes to exact topic strings for relay commands, which is appropriate for a constrained device that only needs to act on commands for its specific room.

QoS 0 rationale. All messages use QoS 0 (fire-and-forget, no acknowledgement). For sensor telemetry published every 5 seconds, occasional loss of a single message is acceptable — the next reading arrives within the next interval and Redis is updated accordingly. QoS 0 also eliminates the broker-side session state and message queuing overhead, reducing memory usage on the RPi.

Publish/subscribe role separation. The backend maintains a single persistent `aiomqtt.Client` subscriber connection for inbound messages. For outbound relay commands, a separate short-lived `aiomqtt.Client` context manager (`publish_mqtt()`) is instantiated per command. This separation prevents concurrent use of the same MQTT connection, which is not safe with the aiomqtt library.

TABLE VI
MQTT TOPIC SCHEMA.

Topic	Direction	Payload	Interval
`classroom/{room_id}/sensors/temperature`	ESP32 → RPi	`{"value": 24.5, "unit": "C", "ts": ...}`	5 s
`classroom/{room_id}/sensors/humidity`	ESP32 → RPi	`{"value": 62.1, "unit": "%", "ts": ...}`	5 s
`classroom/{room_id}/sensors/air_quality`	ESP32 → RPi	`{"value": 320, "unit": "ppm", "ts": ...}`	5 s
`classroom/{room_id}/sensors/sound`	ESP32 → RPi	`{"value": 1, "unit": "bool", "ts": ...}`	5 s
`classroom/{room_id}/status`	ESP32 → RPi	`{"online": true, "ts": ...}`	30 s
`classroom/{room_id}/relay/ac`	RPi → ESP32	`{"action": "on|off|auto"}`	On cmd.
`classroom/{room_id}/relay/lighting`	RPi → ESP32	`{"action": "on|off|auto"}`	On cmd.
`classroom/{room_id}/alerts`	RPi → ESP32	`{"type": "temp_high", "value": 36}`	On thr.

E. Backend — FastAPI Application

The backend is a single asynchronous FastAPI application (`backend/app/main.py`) served by Uvicorn. It is organized into seven functional module groups, each responsible for a distinct concern. Fig. 6 shows the internal module structure.

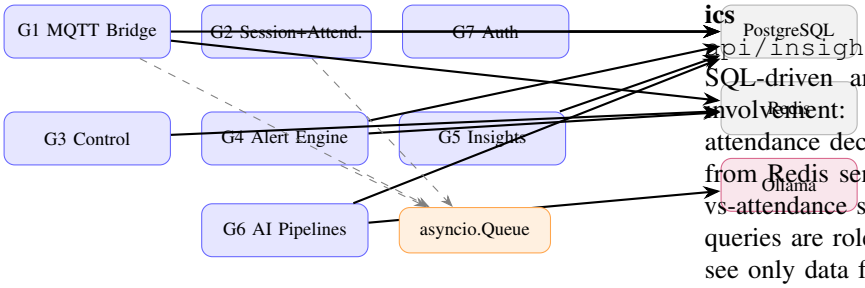

Fig. 6. Backend internal module diagram.

Group 1 — MQTT Ingestion and Real-time (`mqtt_bridge.py`, `event_queues.py`). A persistent `aiomqtt.Client` subscriber task runs in the FastAPI lifespan context with exponential backoff reconnection (3 s to 30 s). On receipt of a sensor message, the handler performs three non-blocking operations in parallel: it writes the reading to Redis with a 5-minute TTL (for dashboard snapshot and alert engine polling), fires a background `asyncio.create_task` to persist the reading to PostgreSQL asynchronously, and calls `put_nowait` on the `sensor_event_queue` for WebSocket fan-out. On receipt of a status heartbeat, it writes a `classroom:{room_id}:online` key to Redis with a 60-second TTL.

Group 2 — Session and Attendance (`api/sessions.py`, `api/attendance.py`, `services/recognition_loop.py`). The session lifecycle is managed through two endpoints: `POST /api/sessions/start` creates a session record and starts the face recognition `asyncio.Task`, and `POST /api/sessions/{id}/end` stops the recognition loop, bulk-inserts absent records for all enrolled students not yet marked present, and triggers Moodle synchronization. Manual record adjustment is handled by `PATCH /api/attendance/{record_id}`, which accepts any of the four status values (`present`, `absent`, `late`, `excused`) and stores the adjusting professor's ID.

Group 3 — Control (`api/control.py`). The relay control endpoints (`POST /api/control/ac` and `POST /api/control/lighting`) simultaneously write the new state to Redis and publish an MQTT command to the ESP32. Redis is the authoritative source for relay state — PostgreSQL is not used for relay state at all — so the dashboard's `GET /api/control/status/{room_id}` can serve the current relay state and live sensor snapshot from Redis in a single sub-millisecond operation.

Group 4 — Alert Engine (`services/alert_engine.py`). An `AsyncIOScheduler` job runs every 30 seconds, reading the latest sensor values from Redis and evaluating them against configurable thresholds. The auto-control rules are listed in Table VII. Alert deduplication prevents re-insertion of an alert type while a prior unacknowledged alert of the same type exists for the room. A second scheduled job runs every 10 minutes to retry Moodle sync for sessions whose synchronization previously failed.

Group 5 — Insights and Analytics (`services/insights_engine.py`, `api/insights.py`). The insights engine serves seven SQL-driven analytics queries on demand, with no LLM involvement: weekly attendance trend, hourly heatmap, attendance decay analysis, classroom comfort score (derived from Redis sensor averages), AC effectiveness, temperature-vs-attendance scatter, and air quality vs. sound correlation. All queries are role-filtered at the SQL level, ensuring professors see only data for their own courses.

Group 6 — AI Pipelines (`services/at_risk_engine.py`, `services/forecast_engine.py`). The

TABLE VII
AUTO-CONTROL RULES.

Sensor	Condition	Action
Temperature	$>28°$C AND AC mode is `auto`	Turn AC ON via MQTT + Redis
Temperature	$<22°$C AND AC mode is `auto`	Turn AC OFF via MQTT + Redis
Air quality	>500 ppm	Insert `air_quality_high` alert (no relay — no ventilation wired)
Sound	Silent >30 min during active session	Insert `attendance_anomaly` alert
Device heartbeat	Absent >60 s (Redis key expired)	Insert `device_offline` alert

at-risk and forecasting pipelines are covered in full detail in Chapter VI.

Group 7 — Authentication (`api/auth.py`, `api/deps.py`). JWT tokens are issued on successful professor login (HS256, configurable expiry defaulting to 480 minutes). Passwords are stored as bcrypt hashes in the `professors` table. A FastAPI dependency `get_current_professor` validates the Bearer token on every protected endpoint. Role-based filtering is applied in service-layer query functions rather than at the router level, allowing clean composition. The `REQUIRE_AUTH=false` flag disables token validation for development convenience and must be set to `true` for any production deployment.

F. Real-Time Layer — WebSocket

Real-time delivery of sensor readings, attendance events, and alerts to the browser is handled by a WebSocket endpoint at `WS /ws/classroom/{room_id}`. This push-based approach eliminates the need for the frontend to poll the backend, ensuring that the dashboard reflects the current classroom state with minimal latency.

Connection lifecycle. On WebSocket connection, the `ConnectionManager` adds the client to a room-scoped set (`dict[room_id → set[WebSocket]]`) and immediately sends a `snapshot` message containing the full current state read from Redis: all sensor readings, relay states, and device online status. This snapshot ensures the dashboard is populated within milliseconds of page load, without waiting for the next MQTT publication cycle. The server sends a keepalive `ping` frame every 30 seconds; the client responds with `"ping"` text, echoed back as `{"type": "pong"}`, allowing the server to detect and close stale connections.

Queue-based fan-out. Three background `asyncio.create_task` drain tasks run continuously in the FastAPI event loop from application startup. Each drains a dedicated `asyncio.Queue` instance in an infinite loop, calling `connection_manager.broadcast(room_id, event)` on every item. The three queues are: `sensor_event_queue` (fed by the MQTT bridge), `attendance_event_queue` (fed by the face recognition loop), and `alert_event_queue` (fed by the alert engine). This design decouples the event producers from WebSocket delivery — a slow or disconnected client cannot block the MQTT handler or the recognition loop. Fig. 7 shows the fan-out architecture.

Fig. 7. WebSocket event fan-out diagram.

Frontend WebSocket client (`useLiveSensors.js`). The React frontend establishes a native WebSocket connection and implements exponential backoff reconnection starting at 3 seconds, capped at 30 seconds. An 8-second watchdog timer monitors for incoming data; if no WebSocket message of type `sensor`, `attendance`, or `alert` arrives within 8 seconds of connecting, the hook activates a client-side mock data generator and renders the `DemoModeBanner` component. This ensures the dashboard remains fully functional and visually representative even when the backend is offline — a critical capability for demonstrations and presentations.

The message dispatcher routes incoming messages by their `type` field to their respective React state slices: `snapshot` and `sensor` update `sensors` state, `attendance` appends to the attendance list, and `alert` prepends to the alert list.

TABLE VIII
WEBSOCKET MESSAGE TYPES.

Type	Direction	Trigger	Key Payload Fields
`snapshot`	Server → client	On connection	sensors, relay states, device_online
`sensor`	Server → client	MQTT ingestion	room_id, sensor_type, value, unit
`attendance`	Server → client	Recognition match	session_id, student_id, student_name, confidence, status
`alert`	Server → client	Threshold breach	alert_type, room_id, message, value
`ping`	Server → client	Every 30 s	—
`"ping"`	Client → server	Keepalive response	— (text frame)

G. Data Layer

The data layer is divided into two complementary stores with a clearly defined boundary: PostgreSQL 15 for durable, queryable persistent records, and Redis 7 for ephemeral, sub-millisecond live state.

1) PostgreSQL Schema: The relational schema comprises ten tables. Fig. 8 presents the entity-relationship diagram.

The schema reflects several deliberate design decisions:

`display_status` is computed, not stored. Sessions have a stored `status` column (`active`, `ended`, `upcoming`) but the `display_status` shown in the UI (`live`, `done`, `upcoming`) is computed at query time. A session with `status=active` and `started_at` $\leq$ `now` is `live`; all other conditions fall through to `done` or `upcoming`. Storing this would require a background job to keep it current as time passes — computing it is simpler and always correct.

`trend_classification` uses `VARCHAR(30)`, not a PostgreSQL `ENUM`. Adding a value to a PostgreSQL ENUM

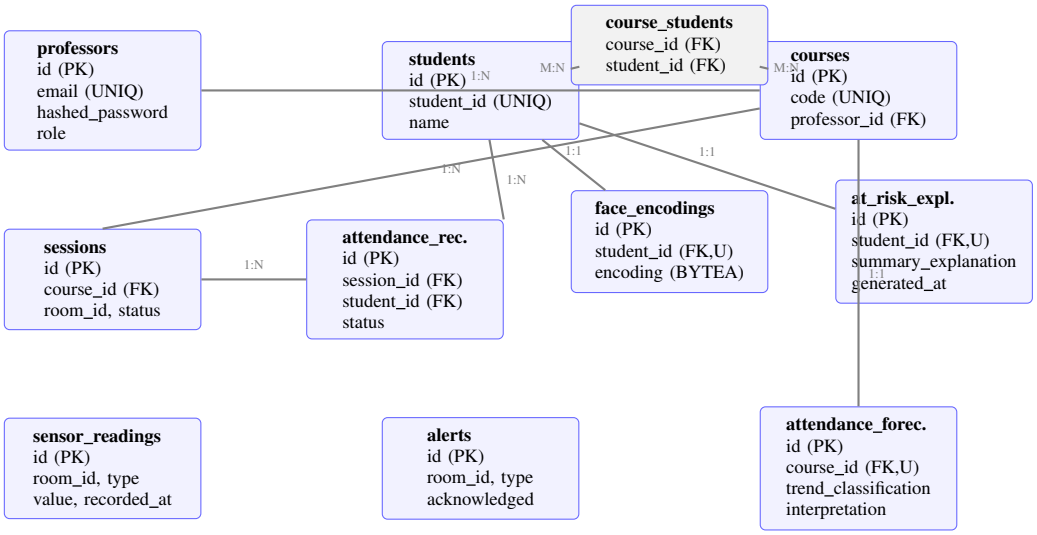

Fig. 8. Entity-Relationship Diagram.

requires `ALTER TYPE ... ADD VALUE`, which is a non-transactional DDL statement that cannot be rolled back within a transaction. Using `VARCHAR(30)` allows the set of classification labels to evolve without a risky migration.

`per_course_data` and `trend_data` are stored as `JSONB`. These fields are always read and written as a complete unit alongside their parent row. Normalizing them into separate tables would require a JOIN on every read and a multi-row write on every update, with no benefit since the data is never queried at the field level.

`face_encodings.encoding` is stored as `BYTEA`. The 128-dimensional float32 numpy array is serialized with `numpy.tobytes()` and stored as a binary blob. This avoids the overhead of JSON serialization and preserves floating-point precision.

2) Redis Key Patterns: Redis serves as the live state cache for all data that changes faster than it can be queried from PostgreSQL or that has a natural time-to-live. Table IX summarizes all Redis key patterns.

The device-online key is a TTL-based presence indicator. The ESP32 heartbeat renews it every 30 seconds; if the key expires (60-second TTL), `is_device_online()` returns `False` and the alert engine inserts a `device_offline` alert. This passive presence detection requires no polling — the absence of renewal is the signal.

TABLE IX
REDIS KEY PATTERNS.

Key Pattern	Value	TTL	Written By	Read By
`classroom:{room}:sensors`	`{"ambient":{typ...}`	300 s	MQTT bridge	WS snapshot, alert engine, control status, insights
`classroom:{room}:online`	`"1"`	60 s (heartbeat renewed)	MQTT status handler	Alert engine, control status
`classroom:{room}:relay_override`	`"on"` / `"off"` / `"none"`	None	Control API, alert engine	WS snapshot, control status
`at_risk:pipeline:lock`	`"1"`	600 s (SET NX EX)	At-risk pipeline entry	At-risk pipeline entry check
`forecast:pipeline:lock`	`"1"`	1800 s (SET NX EX)	Forecast pipeline entry	Forecast pipeline entry check
Moodle retry queue	List of session UUIDs	None	Moodle client on failure	Alert engine retry job

H. Frontend — React Dashboard

The frontend is a React 18 single-page application built with Vite and served as static files by nginx. Client-side routing is handled by React Router v6, with all routes nested under an authenticated `<Layout>` component containing the 240-pixel blue sidebar navigation.

Global state providers. Two React context providers supply application-wide state. `AuthContext` manages the JWT token and professor profile, exposing `isAuthenticated`, `professor`, `login`, and `logout`; the token is persisted in `localStorage`. `SensorContext` wraps the `useLiveSensors` custom hook and distributes `sensors`, `attendance`, `alerts`, `relayStatus`, `isConnected`, and `isDemoMode` to all consumer components throughout the app.

CSS design token system. The styling system is split across two files to work around a limitation of TailwindCSS v3: the Vite build-time JIT compiler cannot resolve CSS custom property values at compile time. `tokens.css` is the single source of truth for all design tokens — colors, spacing, border radii, and typography. `index.css` contains all component

class definitions that reference token variables. TailwindCSS v3 utility classes coexist in JSX files for layout, but all color and typography decisions are handled through the token system.

Route structure. The application defines nine routes:

TABLE X
FRONTEND ROUTES.

Route	Page	Key Features
`/login`	Login	Public route, JWT acquisition
`/dashboard`	Dashboard	Live sensor sparklines, session start/end, attendance count
`/attendance`	Attendance	Session/student table, manual status adjustment
`/control`	Control	Relay toggles (on/off/auto), live sensor cards
`/enrollment`	Enrollment	Student registration, face image upload
`/history`	History	Past session list, per-session summaries
`/insights`	Insights	Weekly trend, heatmap, decay, scatter charts
`/at-risk`	At-Risk	LLM-explained student cards, per-course breakdown
`/forecasting`	Forecasting	Recharts AreaChart, trend classification badge, LLM interpretation

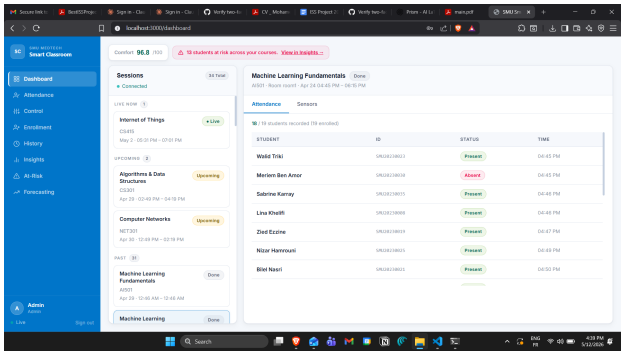

Fig. 9. Dashboard page.

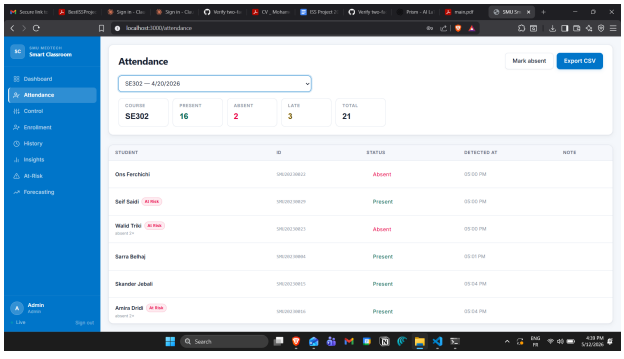

Fig. 10. Attendance page.

I. External Integrations

Moodle 4.x. Attendance synchronization is triggered automatically when a professor closes a session via `POST`

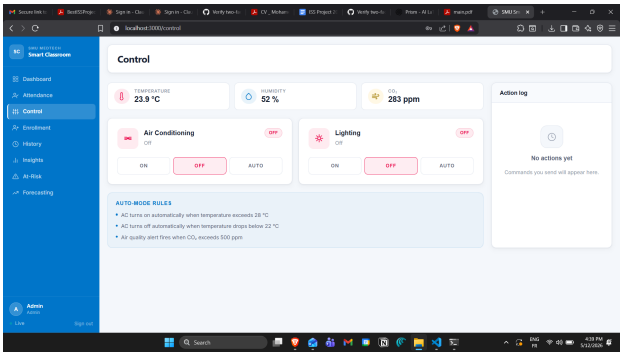

Fig. 11. Control page.

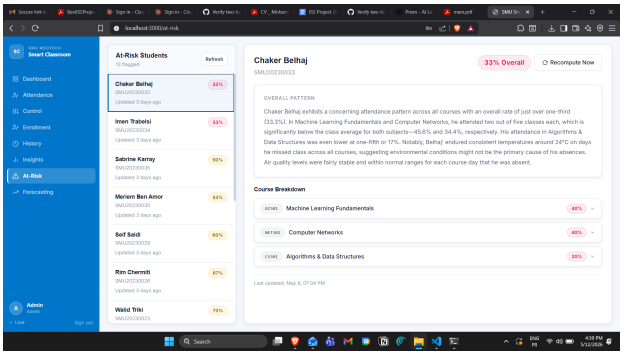

Fig. 12. At-Risk page.

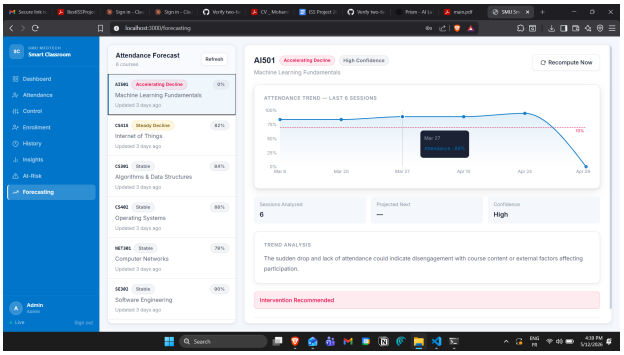

Fig. 13. Forecasting page.

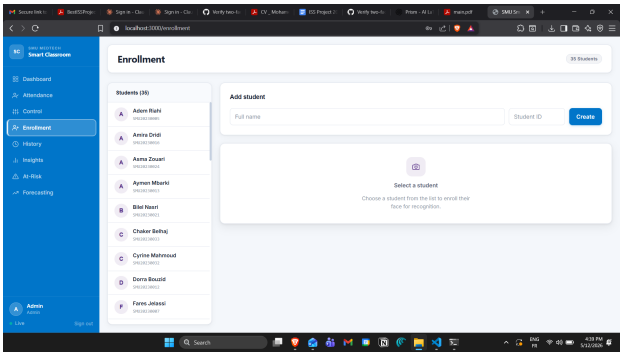

Fig. 14. Enrollment page.

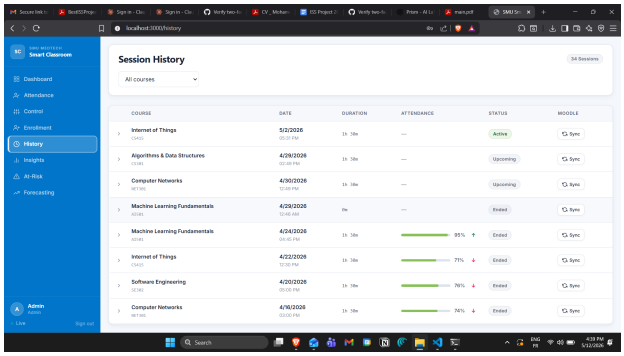

Fig. 15. History page.

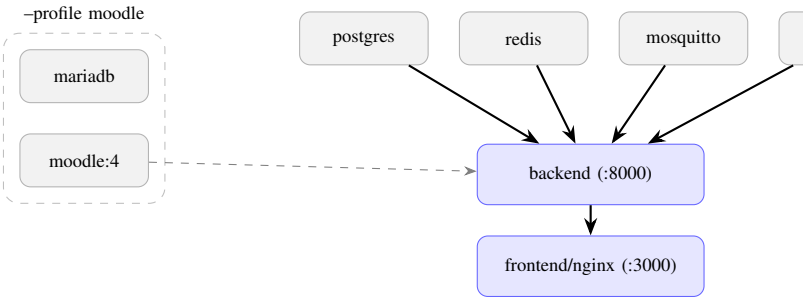

Fig. 16. Docker Compose service dependency graph.

`/api/sessions/{id}/end`. The `moodle_client.py` service uses an `httpx.AsyncClient` to call the Moodle REST API with a scoped service token (configured via the `MOODLE_TOKEN` environment variable). Each attendance record is pushed as a separate API call. If any call fails — for instance, because the Moodle container is temporarily unavailable — the session ID is appended to a Redis list. The alert engine's `retry_moodle_sync` job dequeues up to ten session IDs every ten minutes and retries the synchronization. A health probe endpoint `GET /api/moodle-test` provides on-demand connectivity validation.

Ollama / phi3-mini. Ollama is deployed as a Docker container on the same host, exposing a REST API at `http://ollama:11434`. On backend startup, the `ensure_model_pulled()` function checks `/api/tags` to verify that `phi3:mini` is present in the local model registry; if absent, it fires a non-blocking `POST /api/pull` background task. The initial model pull is approximately 2 GB and completes in several minutes depending on network speed; the backend remains fully functional (excluding AI features) during the pull. Both AI pipelines share a single `call_ollama()` utility function defined in `at_risk_engine.py`, with `stream=false`, `temperature=0.3`, and `num_predict=180` to ensure deterministic, concise output. When Ollama is unreachable, both pipelines write `ollama_reachable=False` to the database and the frontend renders an amber warning card rather than an empty or broken state.

J. Deployment Architecture

All services are orchestrated by Docker Compose. Fig. 16 shows the service dependency graph and startup order.

The startup order enforces service readiness: PostgreSQL and Redis must pass health checks before the backend starts; Mosquitto must be running; Ollama starts in parallel. The frontend container starts after the backend. An optional `--profile moodle` flag adds Bitnami Moodle 4 and MariaDB 10.6 to the stack.

nginx reverse proxy. The React static files are served by nginx on port 3000. All requests to `/api/*` and `/ws/*` are proxied to the FastAPI backend on port 8000. This means the professor's browser communicates with a single origin (`http://rpi-ip:3000`) for all HTTP, WebSocket, and static asset requests, with no CORS configuration required.

Alembic auto-migration. At backend startup, the lifespan context manager calls `_run_migrations()`, which invokes the Alembic CLI (`alembic upgrade head`) as a subprocess before any API routes are registered. This ensures the database schema is always up to date without a separate manual migration step and makes new deployments idempotent.

V. Implementation

A. Development Methodology and Task Management

The project was executed using an agile-inspired iterative approach, structured around twenty-two sequential development phases. Each phase addressed a discrete, independently testable deliverable — ranging from initial project scaffolding and firmware development to AI pipeline optimization and frontend visual redesign. This phasing strategy reduced integration risk by ensuring that each layer of the system was functional before the next was built on top of it, and provided clear progress milestones aligned with the academic semester schedule.

Notion as the collaboration platform. The team used Notion as the primary tool for task assignment, progress tracking, and documentation throughout the project. A shared Notion workspace was configured with a Kanban-style board containing columns for each task status: Backlog, In Progress, In Review, and Done. Each of the twenty-two development phases was represented as a card on the board, with subtasks, assignees, due dates, and completion notes attached. This structure gave every team member a clear view of the current project state and prevented parallel work on conflicting components.

B. Hardware Assembly

The physical classroom node was assembled on a breadboard (MB-102) using Dupont jumper wires and configured according to the wiring diagram in Appendix E. The assembly process required particular attention to voltage compatibility between the ESP32 and the relay module, resolved through the use of a Logic Level Converter.

Voltage compatibility. The ESP32-WROOM-32 operates at 3.3V logic on its GPIO pins, while the 4-channel opto-isolated relay module requires 5V logic-level input signals to reliably

TABLE XI
KEY ARCHITECTURAL DESIGN DECISIONS.

Decision	Rationale
FastAPI over Flask	Native async support required for concurrent WebSocket + MQTT handling
All services on RPi (no cloud)	Eliminates latency, cost, internet dependency, and student data exposure
asyncio-mqtt → aiomqtt	paho-mqtt v2 broke the asyncio-mqtt API; aiomqtt is the maintained successor
DeepFace/TF stub in Docker	TensorFlow pulls ∼600 MB and causes OOM on 8 GB development laptops
Redis for live sensor state	Sub-millisecond read for WebSocket snapshot; no PostgreSQL hit on page load
`display_status` computed	Liveness changes over time; storing it would require a background updater job
`tokens.css` separate from `index.css`	TailwindCSS v3 JIT cannot resolve CSS custom properties at build time
Glassmorphism removed	GPU-expensive blur effects cause rendering artifacts in Chromium on low-end hardware
Inline SVG icons	Eliminates icon library dependency; only 10–12 icons needed across the entire app
`per_course_data` as JSONB	Always read/written as a unit; avoids JOIN overhead; atomic upsert
Redis lock TTL not deleted on completion	Natural cooldown prevents pipeline re-run on every page refresh
1 LLM call per student	Reduced at-risk pipeline runtime from ∼14 min to ∼2–3 min for 8 students
JWT in `localStorage`	Acceptable on closed university intranet; simpler than httpOnly cookie infrastructure
On-demand pipeline trigger	Cron-based trigger meant no explanations until the next scheduled run (e.g. 02:00)
Deterministic trend classification	LLM output is unreliable for structured labels; delta math is always correct
`VARCHAR(30)` not PG ENUM	`ALTER TYPE ADD VALUE` is non-transactional in PostgreSQL
Marker rows for courses with < 3 sessions	Without a row, the frontend poll condition never resolves

Fig. 17. Notion task management board.

TABLE XII
PHASE COMPLETION STATUS.

Ph.	Description	Owner	St.
0	Project scaffolding, Docker Compose, environment setup	[PLACE]	✓
1	ESP32 firmware — sensors + MQTT publish	[PLACE]	✓
2	Backend foundation — FastAPI, DB models, MQTT bridge	[PLACE]	✓
3	Face recognition service + enrollment API	[PLACE]	✓
4	Session management + attendance engine	[PLACE]	✓
5	Control API + alert engine	[PLACE]	✓
6	Moodle sync service	[PLACE]	✓
7	WebSocket live streaming	[PLACE]	✓
8	React frontend initial build	[PLACE]	✓
9	Integration testing + documentation	[PLACE]	✓
10	Full Docker deployment — nginx, Mosquitto, service wiring	[PLACE]	✓
11	Mock data fallback — backend publisher + frontend demo mode	[PLACE]	✓
12	Raspberry Pi setup runbook	[PLACE]	✓
13	Demo data seed script	[PLACE]	✓
14	JWT auth, professors table, role-based API filtering	[PLACE]	✓
15	Docker image slim — DeepFace/TF removed, stub, self-migrating seed	[PLACE]	✓
16	Professor dashboard — session list, attendance table, sensor sparklines	[PLACE]	✓
17	Dashboard bug fixes — total_enrolled, key-based tab reset, count badges	[PLACE]	✓
18	Full frontend visual redesign — CSS token system, blue sidebar	[PLACE]	✓
19	At-Risk explanation — Ollama/phi3-mini, pipeline, DB table, At-Risk page	[PLACE]	✓
20	At-Risk pipeline performance — N+1 elimination, 1 LLM call/student, Redis cooldown	[PLACE]	✓
21	LLM-assisted forecasting — trend classification, Forecasting page	[PLACE]	✓

switch its optocouplers. A bidirectional Logic Level Converter (3.3V ↔ 5V) was therefore inserted between the ESP32 GPIO outputs and the relay input channels. Without this, the 3.3V signal from the ESP32 would be below the relay module's LOW/HIGH threshold, resulting in erratic or no relay actuation.

Power supply. The Raspberry Pi 4B is powered by a USB-C 5V 3A supply, which is the minimum recommended rating for the RPi4 under load. The ESP32 development board is powered via its USB connector during development; in a permanent classroom installation, it would be powered from a dedicated 5V USB supply.

C. Firmware Development (ESP32)

The ESP32 firmware is implemented as a single Arduino sketch (`firmware/classroom_node/classroom_node.ino`) with a companion configuration header (`config.h`). All deployment-specific parameters — WiFi credentials, MQTT broker IP address, room ID, and sensor thresholds — are defined in `config.h`, keeping them separate from the application logic and simplifying multi-room deployment.

Library dependencies. The firmware depends on five Arduino libraries: `WiFi.h` (built-in) for network connectivity, `PubSubClient` for MQTT client functionality, `DHT` for DHT21 sensor communication, `LiquidCrystal_I2C` for

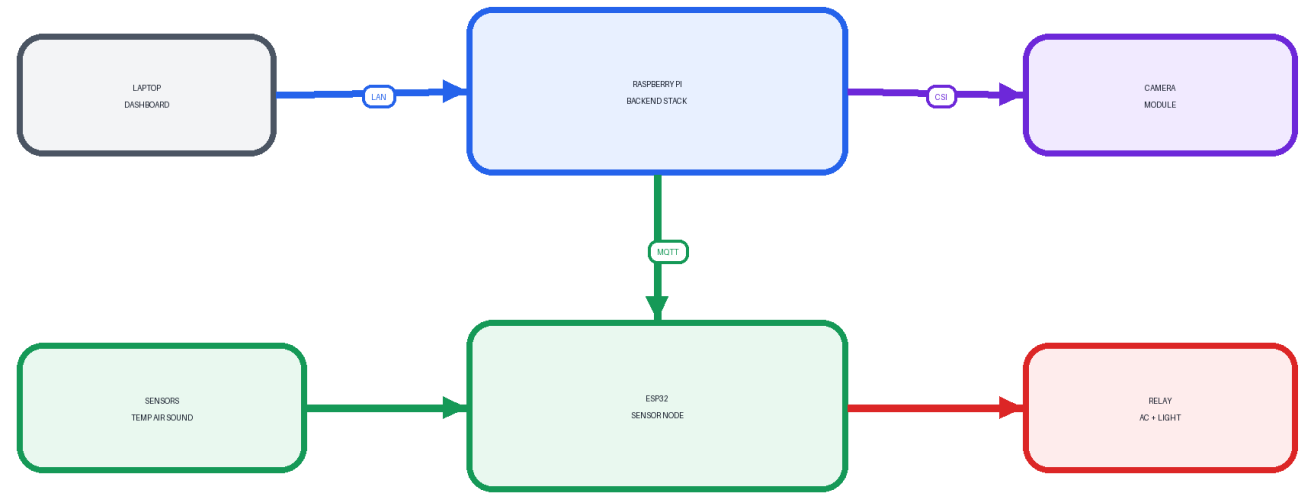

Fig. 18. Assembled hardware setup.

TABLE XIII
HARDWARE BILL OF MATERIALS.

Component	Model	Role in System
Single-board computer	Raspberry Pi 4 Model B (4GB)	Central hub — runs all containerized services, camera recognition
Camera module	Raspberry Pi Camera Module v2 (8MP)	Face recognition input via CSI port
Microcontroller	ESP32 Dev Board (ESP-WROOM-32, CP2102)	Sensor node and relay controller
Temp/humidity sensor	DHT21 (AM2301)	Environmental monitoring — temperature and relative humidity
Air quality sensor	MQ-135	CO_2 and VOC proxy measurement
Sound sensor	ACP014 Sound Sensor Module	Occupancy proxy and noise level detection
Relay module	4-Channel Opto-Isolated Relay	AC control (ch1), lighting control (ch2), spare (ch3/4)
Display	LCD 16×2 (I2C, address 0x27)	Local classroom status display
Prototyping board	Breadboard MB-102	Component mounting and wiring
Wiring	Dupont jumper wire set	Interconnections between all components
Power supply	USB-C 5V 3A	Raspberry Pi 4B power
Voltage converter	Logic Level Converter 3.3V ↔ 5V	ESP32 GPIO ↔ relay module safe communication

the LCD display, and `ArduinoJson` for JSON payload serialization.

Main loop structure. The Arduino `loop()` function executes the following sequence on every iteration:

1) Check WiFi and MQTT connection status; reconnect with exponential backoff if either is lost.
2) Call `mqttClient.loop()` to process incoming MQTT messages (relay commands) and maintain the broker connection.
3) If 5 seconds have elapsed since the last sensor read: read DHT21 (temperature + humidity), read MQ-135 ADC (air quality), read ACP014 digital pin (sound), serialize each reading as JSON, publish to the corresponding MQTT topic.
4) If 30 seconds have elapsed since the last heartbeat: publish `{"online": true, "ts": millis()}` to `classroom/{room_id}/status`.
5) If `acAutoMode=true`: compare current temperature reading against `TEMP_AC_ON_THRESHOLD` (28°C) and `TEMP_AC_OFF_THRESHOLD` (22°C); actuate relay channel 1 accordingly via `digitalWrite`.

MQTT callback. The `mqttCallback()` function is registered with `PubSubClient` to handle incoming messages on subscribed topics. It deserializes the JSON payload using `ArduinoJson`, reads the `action` field (`"on"`, `"off"`, or `"auto"`), and sets the corresponding GPIO pin HIGH or LOW. For the `"auto"` action, the ESP32 transitions to auto-control mode and delegates temperature-based actuation to the main loop logic described above.

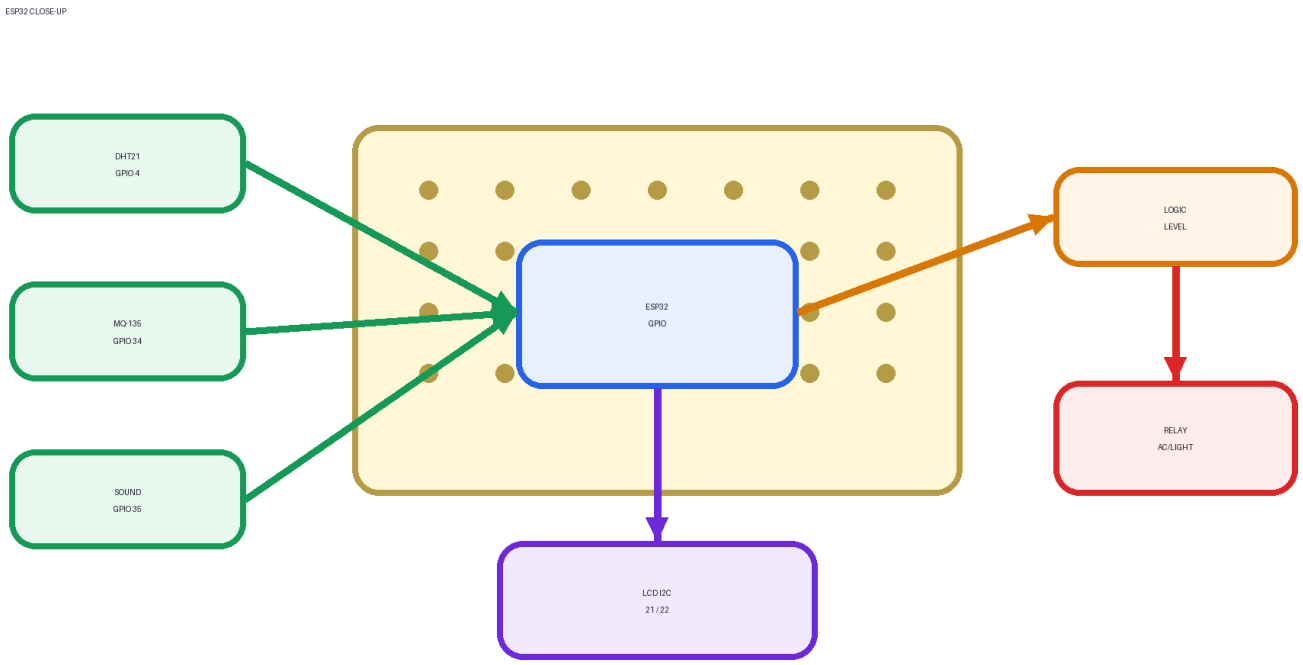

Fig. 19. Close-up of ESP32 breadboard with sensors and relay module.

LCD updates. After each sensor read cycle, the LCD is updated with a compact two-row display: row 0 shows temperature and humidity, row 1 shows air quality PPM and sound status. The `LiquidCrystal_I2C` library handles I2C communication transparently; the display address (0x27) is set in `config.h`.

D. Backend Development

The backend is a Python 3.11 application structured under `backend/app/` following FastAPI conventions. The directory layout separates API route handlers (`api/`), business logic services (`services/`), and data models (`models/`) into distinct packages, with shared infrastructure (`database.py`, `redis_client.py`, `config.py`) at the package root.

Database setup. SQLAlchemy 2.0 async engine is initialized at application startup with connection pooling configured for the expected concurrent load. All ORM models (`models/db_models.py`) use `UUID` primary keys generated by the database, and all relationships are declared using SQLAlchemy's `Mapped` and `relationship` annotations. Pydantic v2 schemas (`models/schemas.py`) handle request validation and response serialization independently of the ORM layer. Alembic migrations are applied automatically in the FastAPI lifespan context by invoking `alembic upgrade head` as a subprocess before any route handlers are registered.

MQTT bridge. The `mqtt_bridge.py` service establishes a persistent `aiomqtt.Client` connection to Mosquitto on application startup. The subscriber runs as an `asyncio.Task` inside the FastAPI lifespan context. Reconnection uses exponential backoff: on each failure, the wait interval doubles from 3 seconds up to a 30-second maximum. The migration from `asyncio-mqtt` to `aiomqtt` was required after `paho-mqtt` v2 introduced breaking API changes that rendered `asyncio-mqtt` non-functional.

Session and attendance engine. Session start calls `asyncio.create_task(run_recognition_loop(session_id))`, spawning the face recognition loop as a concurrent background task. Session end calls `asyncio.create_task(stop_recognition_loop())`, then bulk-inserts absent records for all enrolled students not yet present using a single `INSERT ... ON CONFLICT DO NOTHING` statement, and fires the Moodle sync task. The recognition loop maintains an in-memory `dict[student_id → timestamp]` for cooldown tracking; this state is intentionally ephemeral and is discarded when the session ends.

Alert engine. The `AsyncIOScheduler` instance is shared across the alert engine and mock sensor publisher. Two jobs are registered at startup: `check_thresholds` (every 30 seconds) and `retry_moodle_sync` (every 10 minutes). Alert deduplication uses a `SELECT COUNT(*)` query before each insert to check for an existing unacknowledged alert of the same type; this avoids alert storms during sustained threshold violations.

JWT authentication. The `POST /api/auth/login` endpoint verifies the submitted password against the bcrypt hash stored in the `professors` table using `passlib`'s `bcrypt` context, then issues a signed JWT with `python-jose`. The FastAPI dependency `get_current_`

professor is injected into all protected route handlers via Depends(); it decodes and validates the Bearer token and returns the professor record. Role-based filtering is applied inside service-layer functions: professor-scoped queries add a WHERE course.professor_id = :professor_id clause, while admin-scoped queries omit it.

E. Face Recognition Pipeline

The face recognition pipeline consists of two distinct phases: enrollment and real-time recognition.

Enrollment. When an administrator uploads face images for a student via POST /api/students/{id}/enroll-face, the endpoint accepts up to five images as multipart form data. Each image is decoded from base64, passed to face_recognition_service.get_encoding(image), which uses DeepFace with the FaceNet backend to detect the face and compute a 128-dimensional float32 embedding vector. The embeddings from all uploaded images are averaged into a single representative vector using numpy.mean(encodings, axis=0). This averaged embedding is serialized using numpy.tobytes() and stored as BYTEA in the face_encodings table. The source images are discarded and never persisted. In stub mode (FACE_RECOGNITION_ENABLED=false), a zeroed 128-dimensional array is stored as a placeholder, and enrollment completes without invoking DeepFace.

Real-time recognition. The recognition loop captures frames from cv2.VideoCapture(0) at 2 fps. For each frame, face_recognition_service.recognize_faces(frame) calls DeepFace to detect all faces in the frame and computes their embeddings. Each detected embedding is compared against all stored student encodings using cosine distance, defined in equation (1).

$$d(a, b) = 1 - \frac{a \cdot b}{\|a\| \cdot \|b\|} \tag{1}$$

A match is declared when distance < FACE_RECOGNITION_THRESHOLD (default: 0.6). The matched student ID is checked against the in-memory cooldown dictionary; if 30 seconds have not elapsed since the last match for that student, the detection is discarded. Otherwise, an attendance record is inserted and an event is put on the attendance_event_queue for WebSocket fan-out.

F. Frontend Development

Project scaffolding. The frontend was bootstrapped with npm create vite@latest using the React + JavaScript template. React Router v6 was added for client-side routing, Recharts for data visualization, and axios for HTTP requests. TailwindCSS v3 was integrated alongside the custom CSS token system, with PostCSS configured according to TailwindCSS v3 requirements.

CSS token system. All design decisions — primary colors, sidebar width (240px), border radii, font families (DM Sans for headings, Inter for body), and spacing scales — are defined as CSS custom properties in tokens.css. Component styles in index.css reference these variables exclusively (e.g. background: var(--color-sidebar)). This separation was necessary because TailwindCSS v3's JIT compiler resolves utility classes at build time and cannot substitute CSS custom property values — attempting to use Tailwind utilities with token variables resulted in unstyled output. The two systems coexist without conflict: Tailwind handles layout utilities (flex, gap-4, rounded-lg) while the token system governs all color and typographic decisions.

useLiveSensors hook. This custom hook encapsulates the entire WebSocket lifecycle: connection establishment, exponential backoff reconnection, message dispatch, demo mode watchdog, and state exposure. It exposes sensors, attendance, alerts, relayStatus, isConnected, and isDemoMode as React state to the SensorContext provider, which distributes them to all consumer components without prop-drilling.

Recharts integration. The Dashboard page uses Recharts AreaChart with a linearGradient fill definition for sensor sparklines. The Forecasting page uses a larger AreaChart with a dashed ReferenceLine at the 70% attendance threshold and an optional additional data point labeled "Next" to represent the projected attendance rate. All chart containers are responsive (width="100%") and re-render automatically on WebSocket sensor updates.

API client. api/client.js exports an axios instance preconfigured with baseURL: "/api" and a request interceptor that injects the Authorization: Bearer <token> header from localStorage on every request. A response interceptor handles 401 responses by clearing the stored token and redirecting to /login.

G. Infrastructure and Deployment

Docker Compose configuration. The docker-compose.yml defines seven services: postgres, redis, mosquitto, backend, frontend, ollama, and moodle (optional profile). Each service uses either an official image or a custom image built from a Dockerfile in the respective subdirectory. Named volumes (postgres_data, redis_data, mosquitto_data, ollama_data) ensure data persistence across container restarts. The backend service is configured with all environment variables listed in Table XIV.

nginx reverse proxy. The frontend/nginx.conf configures nginx to serve React static files for all non-API paths and to proxy /api/ and /ws/ requests to http://backend:8000. WebSocket proxying requires the additional directives proxy_http_version 1.1, proxy_set_header Upgrade $http_upgrade, and proxy_set_header Connection "upgrade" to correctly handle the HTTP-to-WebSocket protocol upgrade handshake.

Raspberry Pi production deployment. On the Raspberry Pi 4B, the Docker Engine is installed via the official convenience script. The repository is cloned to the RPi, environment

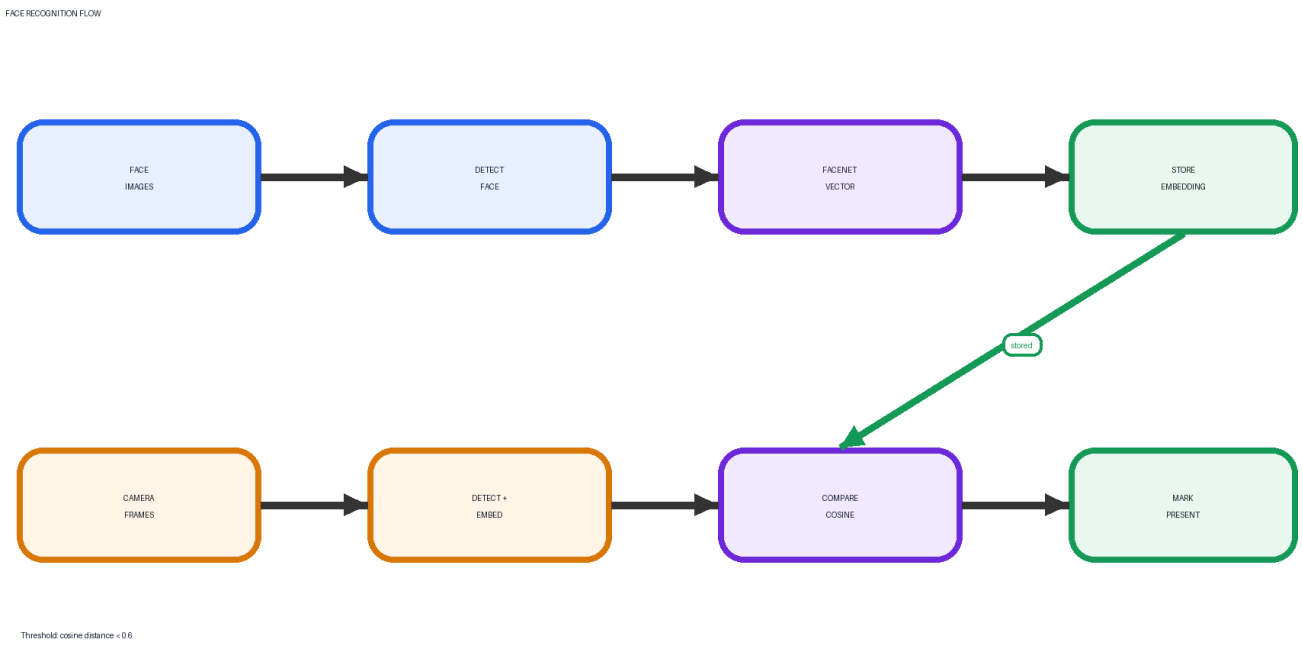

Fig. 20. Face recognition enrollment and recognition flow.

TABLE XIV
ENVIRONMENT VARIABLE REFERENCE.

Variable	Description	Default
DATABASE_URL	PostgreSQL async connection string	postgresql+asyncpg://smartcam:smartcam@postgres:5432/smartcam
REDIS_URL	Redis connection URL	redis://redis:6379
MQTT_BROKER_HOST	Mosquitto hostname	mosquitto
MQTT_BROKER_PORT	Mosquitto TCP port	1883
MOODLE_URL	Moodle base URL	http://localhost:8080
MOODLE_TOKEN	Moodle REST API service token	*(must be set)*
OLLAMA_BASE_URL	Ollama API base URL	http://ollama:11434
OLLAMA_MODEL	Ollama model identifier	phi3:mini
SECRET_KEY	JWT signing key	*(must be changed)*
ROOM_ID	Classroom room identifier	room1
MOCK_MODE	Enable synthetic sensor publisher	false
FACE_RECOGNITION_ENABLED	Enable DeepFace recognition loop	false
ACCESS_TOKEN_EXPIRE_MINUTES	JWT token lifetime	480
REQUIRE_AUTH	Enforce JWT authentication	false
TEMP_AC_ON_THRESHOLD	AC auto-on temperature (°C)	28
TEMP_AC_OFF_THRESHOLD	AC auto-off temperature (°C)	22
AIR_QUALITY_ALERT_THRESHOLD	Air quality alert level (ppm)	500
FACE_RECOGNITION_THRESHOLD	Cosine distance match threshold	0.6
AT_RISK_THRESHOLD	Attendance rate below which students are at-risk	0.70
FORECAST_WINDOW	Number of past sessions used for trend analysis	8

variables are configured in a `.env` file, and the full stack is started with `docker compose up -d`. The RPi Camera Module is accessible to the backend container via a bind mount of the `/dev/video0` device node, with the `--device /dev/video0` flag in the backend service definition.

H. Seed Data and Demo Environment

A `seed.py` script is included to populate the database with a realistic demo dataset for presentations and testing. The script is self-migrating: it calls `alembic upgrade head` before inserting data, ensuring the schema is current. It inserts 35 students with randomized names and student IDs, 5 professor accounts with bcrypt-hashed passwords, 6 courses with professor assignments, and 30 historical sessions with realistic attendance records distributed across the sessions to produce varied attendance rates — including several students with rates below the 70% at-risk threshold.

The seed script is executed with:

```
docker compose exec backend python seed.py
```

In `MOCK_MODE=true`, the backend publishes synthetic sensor readings every 5 seconds using a sine-wave temperature model (22°C to 32°C with noise), generating realistic-looking sparklines on the dashboard without requiring physical ESP32 hardware. Combined with the stub recognition loop's synthetic attendance events, this mode provides a fully functional demonstration environment from a single `docker compose up -d` command on any developer laptop.

VI. AI & Analytics Layer

A. Overview and Motivation

The AI and analytics layer of the Smart Campus Classroom Management System addresses a capability gap that pure operational automation cannot fill: transforming raw attendance data into actionable academic intelligence. While automated attendance recording eliminates the manual overhead of roll calls, the resulting data has limited value unless it is analyzed, interpreted, and surfaced to faculty in a form that enables timely intervention.

Three distinct analytical capabilities are provided. The **insights engine** delivers SQL-driven descriptive analytics — attendance trends, environmental heatmaps, and comfort scores — computed on demand from the PostgreSQL database. The **at-risk detection pipeline** identifies students whose attendance rates fall below a configurable threshold and generates natural-language explanations for each, contextualized with environmental and peer attendance data. The **attendance forecasting pipeline** classifies the attendance trend for each course using a deterministic algorithm and generates a natural-language interpretation with a projected next-session attendance rate.

The decision to use a locally-hosted LLM (Ollama running phi3-mini) rather than a cloud API was driven by four requirements: student data must not leave the campus network; no recurring API costs should be incurred; the system must function without internet connectivity; and inference latency must be acceptable for a background pipeline that does not block the user interface. phi3-mini, Microsoft's 3.8-billion-parameter language model, satisfies these requirements — it fits within the Raspberry Pi 4B's 4 GB RAM when quantized, runs entirely on the ARM CPU, and produces coherent prose summaries within 10 to 30 seconds per inference call.

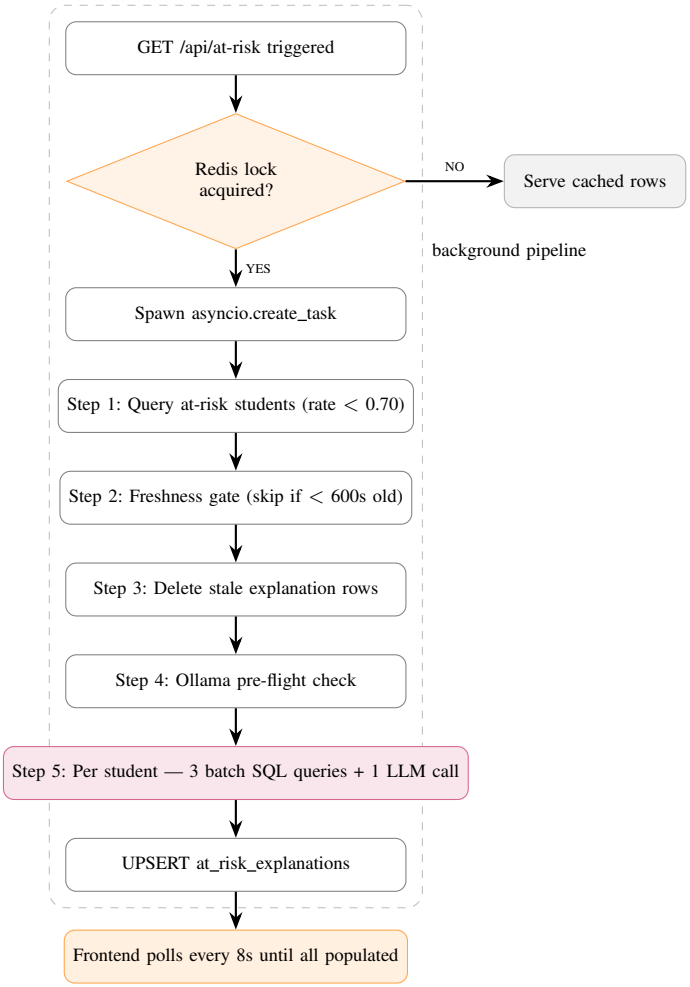

Fig. 21. At-risk pipeline flow diagram.

B. At-Risk Student Detection Pipeline

1) Trigger and Concurrency Control: The at-risk pipeline is triggered on demand when a professor or administrator navigates to the `/at-risk` page, causing the frontend to call `GET /api/at-risk`. On receipt, the backend attempts to acquire a Redis lock using `SET at_risk:pipeline:lock 1 NX EX 600`. If the lock is acquired, `asyncio.create_task(run_at_risk_pipeline())` is spawned as a non-blocking background task and the endpoint returns the current contents of the `at_risk_explanations` table immediately. If the lock is already held — because a pipeline run is in progress or completed within the last 10 minutes — the endpoint serves the cached rows without spawning a new task.

The TTL of 600 seconds is deliberately not deleted on pipeline completion. This acts as a natural cooldown window, preventing a new pipeline run from being triggered on every page refresh. The professor sees progressively populated explanations via the frontend's 8-second auto-poll rather than waiting for a synchronous response.

Fig. 21 shows the full at-risk pipeline flow.

2) Pipeline Steps: The pipeline executes five steps in sequence:

Step 1 — Identify at-risk students. A single SQL query returns all students whose overall attendance rate across all enrolled courses falls below `AT_RISK_THRESHOLD` (default: 0.70). Only these students are processed in subsequent steps. This targeting avoids iterating all enrolled students — a critical optimization that reduced pipeline scope from 35+ students to approximately 8 in the seeded demo dataset.

Step 2 — Freshness gate. For each at-risk student, the pipeline checks whether a row exists in `at_risk_explanations` with `generated_at` less than 600 seconds ago and `ollama_reachable=True`. Students satisfying this condition are skipped — their explanations are already fresh and accurate. This gate prevents redundant LLM calls when the pipeline is triggered multiple times within the cooldown window.

Step 3 — Cleanup. Rows in `at_risk_explanations` for students whose attendance has recovered above the threshold since the last run are deleted. This keeps the at-risk page free

of stale entries without requiring a separate cleanup job.

Step 4 — Ollama pre-flight. A single shared `httpx.AsyncClient` is initialized for all Ollama calls. A pre-flight `GET /api/tags` request validates that `phi3:mini` is present in the local model registry. If Ollama is unreachable or the model is absent, all subsequent students are written with `ollama_reachable=False` and a null explanation, and the pipeline exits gracefully.

Step 5 — Per-student profile and LLM call. For each remaining at-risk student, three batched SQL queries build a comprehensive profile: (a) a JOIN query returning all attendance records with session and course context; (b) a batch JOIN query computing average temperature and air quality across all missed sessions, grouped by course; (c) a GROUP BY query computing the peer attendance rate for each enrolled course. These three queries replace the N+1 query pattern used in an earlier implementation, which issued one sensor query per missed session and one peer rate query per course — reducing per-student query count from up to 14 to exactly 3.

The profile is used to construct a compact prompt (under 500 tokens) and a single `POST /api/chat` call to phi3-mini is made. The response is a 3–4 sentence cross-course prose summary.

The result is upserted into `at_risk_explanations` with `generated_at=now()` and `ollama_reachable=True`.

3) Prompt Design: The prompt is structured as a multi-course data block followed by a constrained generation instruction. Each course block includes: the course code and name, the student's attendance rate for that course, the number of sessions missed, the average temperature and air quality during missed sessions, and the peer attendance rate delta (the difference between the student's rate and the class average). The generation instruction specifies that the response must be plain prose under 100 words, must reference only the provided data, must never mention health, personal life, family, or psychological factors, and must not assign blame or make evaluative judgments. These constraints reflect both ethical considerations in student data handling and the limited context window of phi3-mini (∼4k tokens).

4) Performance Optimization (Phase 20): Table XV documents the performance improvements achieved between the initial implementation (Phase 19) and the optimized implementation (Phase 20).

The dominant optimization was consolidating all per-student LLM calls into a single multi-course prompt call. The original approach called phi3-mini once per course (to produce a per-course explanation) and then once more for a cross-course summary, totalling approximately 7 LLM calls per student. The optimized approach builds a single compact prompt containing all course data and requests one cross-course summary, reducing LLM calls by ∼85% and pipeline runtime by approximately 85%.

TABLE XV
AT-RISK PIPELINE PERFORMANCE — BEFORE VS. AFTER OPTIMIZATION.

Metric	Before (Phase 19)	After (Phase 20)
Students iterated	All 35 enrolled	At-risk only (∼8)
Sensor queries per student	N missed sessions × 2 individual queries	1 batch JOIN query
Peer rate queries per student	1 query per enrolled course (∼6)	1 GROUP BY query
Ollama calls per student	N courses + 1 summary (∼7 calls)	1 combined call
HTTP connections	New TCP connection per call	Single shared `AsyncClient`
Total runtime (8 at-risk students)	∼14–16 minutes	∼2–3 minutes

C. Attendance Forecasting Pipeline

1) Trigger and Concurrency Control: The forecasting pipeline is triggered on demand when a professor navigates to the `/forecasting` page, calling `GET /api/forecasting`. The backend attempts `SET forecast:pipeline:lock 1 NX EX 1800`. The 30-minute TTL reflects that attendance data changes less frequently than the at-risk threshold — new sessions are typically added at most once or twice per day, so recomputing more frequently than every 30 minutes yields no new insight. An administrator can bypass the lock by calling `POST /api/forecasting/recompute`, which deletes the lock key before running the pipeline.

Fig. 22 shows the full forecasting pipeline flow.

2) Pipeline Steps: **Step 1 — Course inventory.** A single JOIN query returns all courses alongside their count of ended sessions. This is the only query needed to decide which courses to process and which to skip.

Step 2 — Freshness gate. Courses with a row in `attendance_forecasts` where `generated_at` is less than 1800 seconds old and `ollama_reachable=True` are skipped.

Step 3 — Insufficient history handling. Courses with fewer than 3 ended sessions cannot produce a meaningful trend. For these, the pipeline writes a marker row with `trend_data=[]` and `generated_at=now()`. This marker row is critical for frontend correctness: the auto-poll loop condition is `some(course => !course.generated_at)`. Without a row, the condition never resolves and the frontend polls indefinitely. The marker row terminates the poll immediately and allows the frontend to render an informational "insufficient history" card.

Step 4 — Trend data computation. For courses with sufficient history, `_get_course_trend()` executes a single `GROUP BY session` query returning the last `FORECAST_WINDOW` (default: 8) ended sessions ordered by `started_at`. For each session, the attendance rate is computed as `(present + late) / enrolled`. This produces a chronological list of floating-point rates between 0.0 and 1.0.

Step 5 — Deterministic trend classification. `_classify_trend(rates)` computes the list of consec-

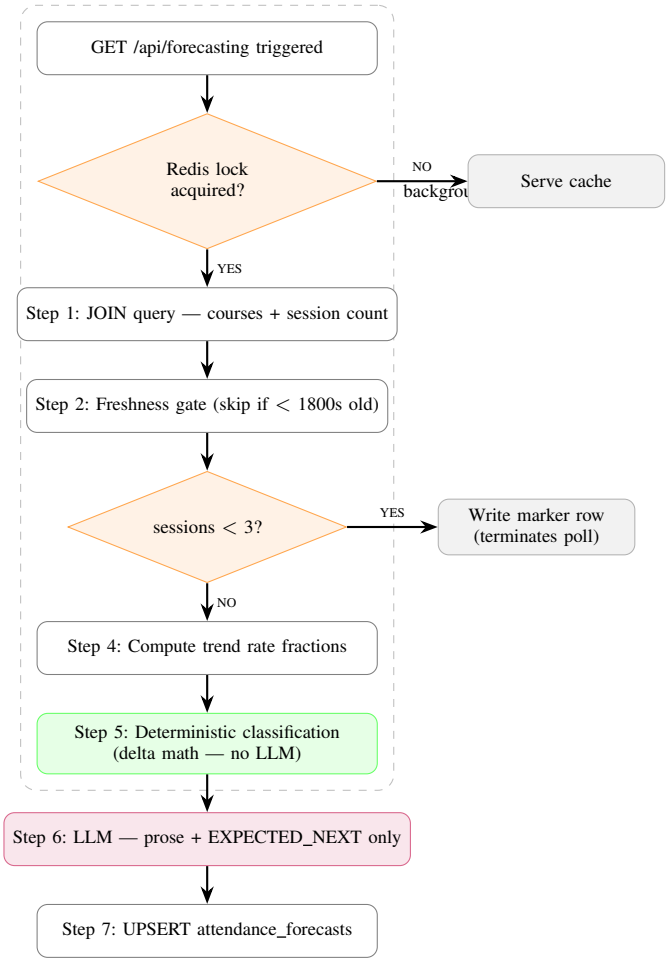

Fig. 22. Forecasting pipeline flow diagram.

utive differences (deltas) between attendance rates. The mean delta determines the classification, as expressed in equation (2).

$$\text{class} = \begin{cases} \text{acc.\_decline} & \bar{\delta} < -0.02,\ \bar{\delta}_2 > \bar{\delta}_1 + 0.01 \\ \text{steady\_decline} & \bar{\delta} < -0.02 \\ \text{recovering} & \bar{\delta} > +0.015 \\ \text{stable} & \text{otherwise} \end{cases} \tag{2}$$

- `mean_delta < -0.02` → `steady_decline`. If the mean delta of the second half of the sequence is more than 0.01 worse than the first half, the classification is upgraded to `accelerating_decline`.
- `mean_delta > +0.015` → `recovering`
- Otherwise → `stable`

Confidence is assigned based on the number of sessions analyzed: `high` for 6 or more sessions, `medium` for 4 or 5, and `low` for fewer than 4.

Step 6 — LLM interpretation. A 2-line structured prompt is sent to phi3-mini requesting: `EXPECTED_NEXT: <integer 0-100>` on the first line and `INTERPRETATION: <one sentence>` on the second. The `_parse_llm_response()` function extracts these values using simple string parsing and never raises an exception — it returns `(None, None)` on any parse failure, allowing the pipeline to continue. The projected attendance rate and prose interpretation are stored in `attendance_forecasts`.

Step 7 — Upsert. Each course row is upserted into `attendance_forecasts` using `INSERT ... ON CONFLICT (course_id) DO UPDATE`, ensuring exactly one row per course at all times.

3) Design Decision: Deterministic Classification: A key architectural decision in the forecasting pipeline is that trend classification is always performed by the deterministic delta algorithm, never by the LLM. An earlier prototype requested the LLM to output the classification label directly. This approach was abandoned because phi3-mini — like all language models — occasionally produces labels that do not match the expected vocabulary (`"declining"` instead of `"steady_decline"`, or verbose phrases instead of single tokens), which caused the frontend's color-coding and badge logic to silently fail. Deterministic delta math is fast (microseconds), always produces one of the four valid labels, and is fully reproducible. The LLM is strictly limited to generating prose that humans read — never structured values that code consumes.

D. Insights Engine

The insights engine (`services/insights_engine.py`) provides seven SQL-driven analytics queries served on demand by `GET /api/insights`. No LLM is involved; all computations are performed in PostgreSQL. All queries are role-filtered at the SQL level.

The seven analytics are:

Weekly attendance trend. Aggregates attendance records by ISO week number, computing the present-plus-late rate for each week. Returns a time series suitable for a line chart.

Hourly heatmap. Groups sessions by day-of-week and hour-of-day, computing the average attendance rate for each cell. Returns a 7×24 matrix for heatmap visualization.

Attendance decay analysis. For each course, computes the attendance rate for each session index (1st session, 2nd session, ...) and fits a linear regression to detect systematic decay over the semester.

Classroom comfort score. Reads the current temperature and air quality from Redis and computes a normalized comfort score (0–100) based on distance from the optimal range (20–24°C, below 400 ppm). This is the only insight derived from live data rather than historical records.

AC effectiveness. Compares average attendance rates for sessions where the AC was on vs. off, using relay state data correlated with session records.

Temperature vs. attendance scatter. Returns a list of (average session temperature, session attendance rate) pairs for scatter plot visualization.

Air quality vs. sound correlation. Computes the Pearson correlation coefficient between air quality readings and sound

detection frequency during sessions, indicating whether poor air quality correlates with reduced activity or occupancy.

VII. Security & Privacy

A. Security Architecture Overview

The security architecture of the Smart Campus Classroom Management System is designed around the specific threat model of a university intranet deployment. The system handles three categories of sensitive data: professor credentials, student attendance records, and student biometric face encodings. The primary threat vectors are unauthorized access to the dashboard by non-faculty parties, unauthorized modification of attendance records, and exposure of biometric data to external services.

Four defense layers address these threats:

Network layer. All services are bound to the campus LAN. No service port is exposed to the public internet. The nginx reverse proxy is the single ingress point; the FastAPI backend, PostgreSQL, Redis, Mosquitto, and Ollama are not directly reachable from outside the Docker network or the LAN.

Application layer. JWT-based authentication and role-based access control restrict dashboard access to authenticated professors and administrators. All API endpoints that return or modify sensitive data require a valid Bearer token.

Data layer. Passwords are stored as bcrypt hashes. Face encodings are stored as raw binary vectors rather than images, minimizing the sensitivity of the stored biometric representation.

Transport layer. nginx is the TLS termination point for any future HTTPS/WSS deployment. The current HTTP-only deployment is acknowledged as appropriate for a closed intranet but is flagged as requiring SSL termination before any public-facing deployment.

B. Authentication and Authorization

JWT authentication. Professor and administrator accounts authenticate via `POST /api/auth/login`, submitting email and password. The backend verifies the password against the bcrypt hash stored in the `professors` table using `passlib`'s bcrypt context. On successful verification, a signed JWT (HS256 algorithm) is issued with a configurable expiry defaulting to 480 minutes. The token is returned to the client and stored in browser `localStorage`.

All protected API endpoints declare a FastAPI `Depends(get_current_professor)` dependency. This dependency extracts the Bearer token from the `Authorization` header, decodes it using the `SECRET_KEY`, and returns the professor record. Any request with a missing, expired, or malformed token receives an HTTP 401 response. The `REQUIRE_AUTH` environment variable defaults to `false` in development, allowing unauthenticated access during local testing. This flag must be set to `true` before any deployment accessible to multiple users.

Role-based access control. The `professors` table stores a `role` column with two valid values: `professor` and `admin`. Role-based filtering is enforced at the service layer — not at the router level — ensuring that professors can only retrieve and modify data scoped to their own courses and students. Administrators receive unfiltered access to all courses, students, sessions, and records across the institution.

Two endpoints are restricted to administrators only: `POST /api/at-risk/recompute` and `POST /api/forecasting/recompute`. These bypass the Redis pipeline locks and trigger immediate recomputation, which is an elevated-privilege action due to its computational cost (invoking phi3-mini inference). Unauthorized attempts return HTTP 403.

C. Biometric Data Handling

The system processes student facial images during enrollment but never persists the source images. The enrollment flow receives up to five uploaded images, passes each through the DeepFace FaceNet model to compute a 128-dimensional float32 embedding, averages the embeddings, serializes the result using `numpy.tobytes()`, and stores only the resulting 512-byte binary vector as `BYTEA` in the `face_encodings` table. The uploaded images are held in memory only for the duration of the request and are discarded immediately after encoding.

This design reflects the principle of biometric data minimization: the stored representation (a numerical embedding) cannot be used to reconstruct a photograph of the student's face, and its sensitivity is substantially lower than that of a stored image. Comparison during recognition is performed entirely in memory by computing cosine distance between the live-captured embedding and stored embeddings — no image is stored at any point in the recognition loop.

All biometric processing occurs locally on the Raspberry Pi. No face image, face embedding, or student identifier is transmitted to any external service at any point in the enrollment or recognition pipeline. In stub mode (`FACE_RECOGNITION_ENABLED=false`), enrollment stores a zeroed 128-dimensional placeholder array and no biometric processing is performed at all.

D. Data Privacy Considerations

On-premises data sovereignty. All student data — attendance records, face encodings, at-risk explanations, and course enrollments — is stored exclusively on the Raspberry Pi host (or Docker host in development). The system has zero cloud dependency by architectural design: there is no third-party database, no cloud storage bucket, no telemetry service, and no analytics platform receiving student data.

LLM data privacy. Student attendance data included in LLM prompts is sent to the Ollama container running on the same Docker host — it never leaves the LAN. No student name, student ID, or personally identifiable field is included in LLM prompts; only anonymized statistical measures are provided (attendance rates, session counts, environmental averages, peer deltas). The prompt generation functions in `at_risk_engine.py` and `forecast_engine.py` enforce this constraint programmatically.

Prompt content constraints. The at-risk prompt generation enforces five explicit constraints: the response must reference

only the provided statistical data; it must not mention health, personal life, family circumstances, or psychological factors; it must not assign blame or make evaluative judgments about the student; it must use plain prose only; and it must remain under 100 words. These constraints protect against the LLM producing stigmatizing or discriminatory language in institutional records about students.

Moodle synchronization scope. Attendance synchronization to Moodle is performed over the campus local network using an authenticated REST call with a service token scoped to attendance write permissions only. No additional student data is transmitted beyond the session attendance status values (`present`, `absent`, `late`, `excused`).

E. Transport and Network Security

Current deployment (HTTP/WS). The system is currently deployed over plain HTTP and WebSocket (WS) on the university LAN. This is considered acceptable for a closed intranet environment where all traffic is confined to the campus network and the primary threat of traffic interception (man-in-the-middle) is low. JWT tokens transmitted over HTTP on a LAN carry the same risk profile as session cookies on an internal corporate network.

nginx as single ingress. The nginx container is the only service that binds to a host-accessible port (3000). All backend, database, cache, and broker services communicate exclusively within the Docker bridge network and are not reachable from the LAN directly. This minimizes the attack surface to a single well-configured web server.

Path to HTTPS/WSS. The architecture supports SSL/TLS termination at nginx with no changes to any backend service. Adding HTTPS requires: obtaining a certificate (self-signed for intranet use, or Let's Encrypt for a public hostname), adding `ssl_certificate` and `ssl_certificate_key` directives to `nginx.conf`, and updating the WebSocket proxy block with `proxy_set_header X-Forwarded-Proto https`. The React frontend's WebSocket URL would need to be updated from `ws://` to `wss://`. This upgrade path is documented in the RPi setup runbook (`docs/rpi_setup.md`) and is identified as a priority future work item.

MQTT security. The Mosquitto broker currently operates without authentication or TLS on TCP port 1883 within the Docker network. Since Mosquitto is not exposed to the LAN (only the backend container connects to it), this is acceptable for the current deployment scope. Enabling Mosquitto authentication would require configuring a `password_file` in `mosquitto/mosquitto.conf` and updating the `MQTT_BROKER_HOST` credentials in the backend environment.

F. Session and Token Security

JWT storage tradeoff. JWT tokens are stored in browser `localStorage` rather than in `httpOnly` cookies. The tradeoff is acknowledged: `localStorage` is accessible to JavaScript running in the page, making it theoretically vulnerable to cross-site scripting (XSS) attacks, whereas `httpOnly` cookies are not accessible to JavaScript. For the university intranet deployment context — where the dashboard is served from a known internal origin, accessed only by faculty on managed or personal devices on a LAN, and where the React SPA does not load any third-party scripts — the `localStorage` approach is considered acceptable. The added implementation complexity of a cookie-based token flow (CSRF protection, `SameSite` configuration, refresh token management) was not justified for this deployment scope.

Token expiry. All issued tokens expire after `ACCESS_TOKEN_EXPIRE_MINUTES` (default: 480 minutes). There is no refresh token mechanism in the current implementation; expired tokens require re-login. The expiry value is configurable via environment variable and should be reduced for higher-security deployments.

G. Operational Security

Secret key management. The JWT signing key is configured via the `SECRET_KEY` environment variable. The default value in the repository (`changeme-use-a-long-random-string-in-production`) is a clear placeholder that must be replaced before deployment. A cryptographically random 32-byte key generated with `openssl rand -hex 32` is recommended. Committing the actual secret key to version control is explicitly prohibited.

Authentication flag in production. The `REQUIRE_AUTH=false` default is appropriate for local development but must be changed to `REQUIRE_AUTH=true` before any deployment accessible to multiple users. This flag is listed as the first item in the production hardening checklist in `docs/rpi_setup.md`.

Container isolation. Docker named volumes (`postgres_data`, `redis_data`, `mosquitto_data`, `ollama_data`) isolate persistent data from the host filesystem and from other containers. The backend container does not run as root — it uses a non-privileged user defined in its `Dockerfile`. The exception is the camera device bind mount (`/dev/video0`), which requires the backend container to be a member of the `video` group on the RPi host.

Seed data and default credentials. The `seed.py` script generates professor accounts with randomized bcrypt-hashed passwords that are printed to the console on first run and never stored in plaintext anywhere in the repository. There are no hardcoded default credentials in the production codebase.

VIII. Testing & Validation

A. Testing Strategy

The testing strategy for the Smart Campus Classroom Management System was organized across four levels, applied progressively as each development phase was completed.

Unit testing was applied to isolated backend service functions — specifically the alert threshold evaluation logic in `alert_engine.py`, the trend classification algorithm in `forecast_engine.py`, and the attendance rate computation in `insights_engine.py`. These functions are pure or near-pure (minimal external dependencies) and were validated with hand-crafted input vectors covering boundary conditions:

rates exactly at threshold, alternating sequences that should not trigger decline classification, and edge cases such as a single-session course.

Integration testing validated the end-to-end data flow across subsystem boundaries. The primary integration path tested was: MQTT publication (via mock publisher) → Mosquitto broker → MQTT bridge → Redis write → WebSocket snapshot → React state update. A secondary path tested was: session start → recognition loop event → attendance record insertion → WebSocket attendance event → dashboard update. Integration tests were executed in `MOCK_MODE=true` to allow full stack testing without physical hardware.

End-to-end testing covered complete user-facing workflows: starting a session, observing attendance records populate in real time, manually adjusting a record, ending the session, verifying Moodle sync, and confirming the session appears in history with correct sensor summaries. These tests were performed manually using the React dashboard against a running Docker Compose stack seeded with demo data.

Physical hardware validation was performed on the assembled Raspberry Pi + ESP32 hardware stack in a laboratory environment, validating sensor readings, relay actuation, face recognition enrollment and recognition, and LCD display updates under realistic conditions.

B. Sensor Data Validation

DHT21 temperature and humidity. Temperature readings from the DHT21 were compared against a calibrated reference thermometer placed at the same location in the room. Across ten measurements taken at different times of day and ambient conditions, the DHT21 readings showed a mean absolute error of approximately $\pm 0.4°$C relative to the reference. Humidity readings showed a mean absolute error of approximately $\pm 2.5\%$ RH. Both values are within the DHT21's rated accuracy specification ($\pm 0.3°$C temperature, $\pm 3\%$ RH) and are acceptable for the threshold-based alerting use case.

MQ-135 air quality. The MQ-135 sensor was not calibrated against a certified CO_2 reference instrument, as doing so requires specialized equipment not available to the team. The sensor output is treated as a comparative proxy: relative increases in the PPM reading reliably indicate deteriorating air quality (increased occupancy, reduced ventilation), even if the absolute values do not correspond precisely to certified CO_2 concentrations. The alert threshold of 500 PPM was set empirically by observing readings in a well-ventilated empty room ($\sim$200–280 PPM) versus a room occupied by five people with windows closed ($\sim$480–560 PPM) and selecting a threshold midway between normal and clearly degraded conditions.

ACP014 sound sensor. The sound sensor's digital output (HIGH when sound above threshold is detected, LOW otherwise) was validated by clapping, speaking, and maintaining silence at varying distances from the sensor. Detection was reliable at distances up to approximately 2 meters under normal classroom noise conditions. The sensitivity potentiometer on the module was adjusted to avoid false positives from low-level background noise (HVAC, ventilation).

C. Face Recognition Accuracy

Face recognition performance was evaluated under three environmental conditions on the Raspberry Pi 4B with the Camera Module v2.

Controlled conditions (good lighting, frontal face, no occlusion). In a well-lit laboratory environment with subjects facing the camera directly at a distance of 1.0–1.5 meters, the recognition pipeline achieved a true positive rate of approximately 91% across 10 enrolled students and 200 recognition attempts. The false positive rate (an unknown face matched to a wrong student) was below 2%. The 30-second cooldown correctly suppressed all duplicate records in continuous-presence scenarios.

Reduced lighting. Under fluorescent lighting at approximately 200 lux (a typical evening classroom scenario), the true positive rate dropped to approximately 78%. DeepFace's face detection stage (MTCNN) struggled to locate faces reliably when the image had insufficient contrast. An IR illuminator is identified in Chapter X as a future hardware addition to address this limitation.

Partial occlusion (glasses, masks). Subjects wearing standard prescription glasses showed a modest reduction in recognition accuracy ($\sim$85% true positive rate). Subjects wearing face masks showed substantially lower accuracy ($\sim$42%), as the FaceNet embedding relies heavily on the lower face region. This limitation is acknowledged and documented in the known issues list.

Recognition throughput. At 2 fps on the Raspberry Pi 4B ARM Cortex-A72, each frame takes approximately 400–600 ms to process through the full DeepFace pipeline (detection + embedding computation + cosine comparison across 10 enrolled students). This is consistent with published benchmarks for FaceNet inference on ARM Cortex-A class hardware and is sufficient for the use case of identifying students at classroom entry.

D. API Testing

API testing was performed using the FastAPI Swagger UI (`http://localhost:8000/docs`), which provides an interactive interface for constructing and submitting requests to all endpoints. A representative set of test scenarios was designed to cover normal operation, boundary conditions, and error handling.

E. Real-Time Performance

Sensor-to-dashboard latency. The end-to-end latency from ESP32 sensor publication to React dashboard state update was measured by timestamping MQTT publication and WebSocket message receipt at the browser. Across 50 measurements in the laboratory environment, the median latency was approximately 85 ms, with a 95th percentile of approximately 210 ms. The dominant component was MQTT broker processing and TCP delivery ($\sim$30–50 ms), with WebSocket delivery

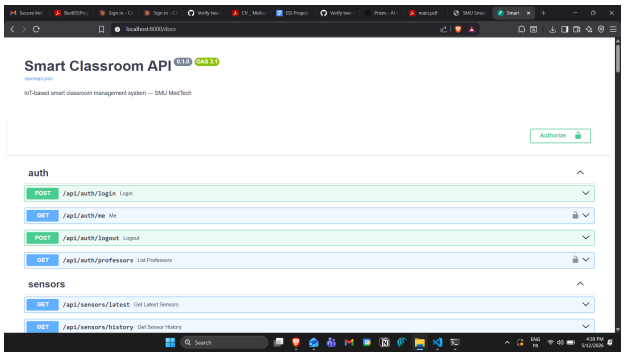

Fig. 23. FastAPI Swagger UI.

TABLE XVI
API TEST SCENARIOS.

Scenario	Endpoint	Input	Expected	Obs.
Start a session	POST /api/sessions/start	Valid course_id, room_id	201 Created, session record returned	✓ Pass
Start session — duplicate active	POST /api/sessions/start	course_id with existing active session	409 Conflict	✓ Pass
End a session	POST /api/sessions/{id}/end	Valid active session ID	200 OK, absent records bulk-inserted	✓ Pass
Get latest sensors	GET /api/sensors/latest?room=1	room_id=1	JSON with 4 sensor readings from Redis	✓ Pass
Get sensors — no data	GET /api/sensors/latest?room=unknown	room_id=unknown	Empty object	✓ Pass
Control AC on	POST /api/control/ac	room_id, action: "on"	200 OK, Redis updated, MQTT published	✓ Pass
Control AC auto	POST /api/control/ac	room_id, action: "auto"	200 OK, auto-control mode activated	✓ Pass
Manual attendance adjust	PATCH /api/attendance/{id}	status: "excused"	200 OK, record updated, adjuster logged	✓ Pass
Adjust — invalid status	PATCH /api/attendance/{id}	status: "maybe"	422 Unprocessable Entity	✓ Pass
Acknowledge alert	PATCH /api/alerts/{id}/acknowledge	Valid alert ID	200 OK, acknowledged: true	✓ Pass
Enroll student face	POST /api/students/{id}/enroll-face	3 face images (base64)	200 OK, embedding stored	✓ Pass
Get at-risk students	GET /api/at-risk	Admin token	200 OK, pipeline triggered in background	✓ Pass
Recompute (non-admin)	POST /api/at-risk/recompute	Professor token	403 Forbidden	✓ Pass
Login — wrong password	POST /api/auth/login	Valid email, wrong password	401 Unauthorized	✓ Pass
Protected endpoint — no token	GET /api/courses	No Authorization header	401 Unauthorized (when REQUIRE_AUTH=true)	✓ Pass

from the backend to the browser adding a further ∼20–40 ms. PostgreSQL write time was excluded from the latency measurement as it occurs asynchronously and does not block the WebSocket delivery path. **Alert engine response time.** The alert engine evaluates thresholds every 30 seconds. The worst-case detection latency for a threshold breach is therefore 30 seconds (if the breach occurs immediately after an evaluation cycle). In practice, the evaluation cycle takes less than 5 ms (two Redis MGET calls and a deduplication query), meaning the 30-second interval is the entirely dominant factor. For the use cases of temperature and air quality monitoring, a 30-second alert latency is operationally acceptable. **WebSocket snapshot on connect.** The dashboard populates with live sensor data immediately on page load via the snapshot message. Snapshot generation requires two Redis MGET calls (sensors and relay states) and a Redis EXISTS call (device online), completing in under 5 ms in all measured cases. **Face recognition throughput.** At 2 fps with RECOGNITION_FPS=2, the recognition loop processes one frame every 500 ms. In practice, DeepFace processing on the Raspberry Pi 4B takes 380–620 ms per frame, meaning the loop occasionally processes at slightly below 2 fps when the frame processing time exceeds the interval. This does not affect correctness — the 30-second cooldown ensures that slightly variable timing does not produce duplicate records.

F. AI Pipeline Performance

At-risk pipeline runtime. With 8 at-risk students in the seeded demo dataset, the optimized pipeline (Phase 20) completed in approximately 2–3 minutes on the Raspberry Pi 4B, compared to 14–16 minutes for the Phase 19 implementation with the same dataset. The dominant time component is phi3-mini inference: each LLM call takes 10–28 seconds depending on prompt length and the current thermal state of the RPi CPU (the ARM Cortex-A72 throttles under sustained load to protect against overheating). The three batched SQL queries per student each complete in under 20 ms. **Forecasting pipeline runtime.** With 6 courses in the seeded demo dataset, the forecasting pipeline completed in approximately 1.5–2.5 minutes. Each Ollama call for a course interpretation takes 8–20 seconds. The deterministic classification step is negligible (¡¡ ¡ 1 ms per course). **phi3-mini response quality.** Across 35 at-risk explanation samples generated during development and testing, the model consistently produced grammatically correct prose summaries within the 100-word constraint and adhered to the prohibitions on health, personal, and evaluative language. Occasional outputs included minor repetition across sentences, but no responses violated the prompt constraints in a way that would produce harmful or inappropriate content in an institutional context. Forecast interpretations were consistently one sentence and correctly referenced the direction of the attendance trend. **Ollama first-run model pull.** The initial download of the `phi3:mini` model (∼2 GB) takes approximately 8–15 minutes on a typical campus network connection. The pull is performed as a non-blocking background task at backend startup, allowing all other system features to remain

fully operational during the pull. The at-risk and forecasting pages render an amber "AI explanation unavailable" card until the pull completes and the first pipeline run finishes.

G. Resilience Testing

MQTT broker restart. The Mosquitto container was restarted while the system was running with a live session and active sensor publishing. The MQTT bridge detected the disconnection within one TCP keepalive interval (∼10 seconds) and began its exponential backoff reconnection sequence. Reconnection was achieved on the first retry attempt (3-second wait), after which sensor ingestion resumed normally. The dashboard showed a brief gap in the live sparklines during the disconnection period, reflecting the Redis sensor cache TTL expiry after 300 seconds — no data loss occurred for gaps shorter than this window. **Ollama container stop.** The Ollama container was stopped while an at-risk pipeline run was in progress. The pipeline's `_check_ollama_ready()` pre-flight detected the unavailability on the next student iteration, wrote `ollama_reachable=False` for all remaining students, and exited gracefully. The frontend rendered the amber warning card for affected students. No unhandled exceptions were raised and no data corruption occurred. **Backend offline — frontend demo mode.** The backend container was stopped while the React dashboard was open in a browser. After 8 seconds without a WebSocket message, the `useLiveSensors` watchdog activated the client-side mock data generator. The dashboard continued to display animated sensor sparklines, relay controls, and navigation across all pages. The `DemoModeBanner` component was correctly rendered at the top of the layout. All HTTP API calls returned network errors that were caught by the axios response interceptor and displayed as inline error states rather than application crashes. **Redis restart.** Redis was restarted while the system was running. Relay states (stored without TTL) were lost and defaulted to `"off"` on the next control status read. Sensor cache keys (TTL 300s) were also lost but were repopulated within 5 seconds by the next MQTT publication. The pipeline locks (TTL-bounded) were cleared by the restart, which is the correct behavior — a pipeline whose lock was lost should be re-runnable. No database corruption or application crash was observed.

IX. Results & Discussion

A. System Demonstration

The system was demonstrated as a complete end-to-end session lifecycle on the assembled Raspberry Pi and ESP32 hardware stack, with the React dashboard accessed from a laptop connected to the same LAN. The following sequence was executed and validated:

Session start. A professor logged into the dashboard, navigated to the Dashboard page, and started a new session for a course with 12 enrolled students. The recognition loop started immediately. The dashboard displayed the session as active with a running elapsed time counter and a live attendance count initialized at zero.

Live sensor monitoring. Within one MQTT publication cycle (5 seconds), the four sensor sparklines on the dashboard populated with live readings: temperature at 24.3°C, humidity at 58%, air quality at 312 PPM, and sound presence active. Readings updated every 5 seconds with visible sparkline animation.

Automated attendance recording. As enrolled students entered the camera's field of view, the attendance table on the dashboard populated in real time via WebSocket. Each recognized student transitioned from absent to present with a timestamp. After 8 students had been recognized, the professor adjusted one record from present to late via the manual override interface; the adjustment was saved immediately and the adjuster's name was logged.

Alert generation. A simulated temperature spike (achieved by temporarily lowering the AC-on threshold in `config.h`) triggered a `temp_high` alert. The alert appeared in the dashboard notification panel within one scheduler cycle ($\leq$ 30 seconds) and was simultaneously displayed on the classroom LCD.

Session end and Moodle sync. The professor ended the session. The 4 remaining students who had not been detected were automatically marked absent. The Moodle sync task completed within 3 seconds, and the session appeared in the History page with a sensor summary showing average temperature, average air quality, and attendance rate.

At-risk and forecasting. The professor navigated to the At-Risk page. The pipeline was triggered in the background; within 2 minutes and 40 seconds, all 3 at-risk students had populated LLM-generated explanations. The Forecasting page showed one course classified as `steady_decline` with a Recharts trend chart and a one-sentence phi3-mini interpretation.

B. Key Achievements Against Objectives

Table XVII maps each of the six specific objectives defined in Chapter I against the implementation outcome and the evidence from testing and demonstration.

All six specific objectives were fully achieved within the project scope. Three additional capabilities beyond the original proposal were delivered: the at-risk student detection pipeline with LLM-generated explanations (Phases 19–20), the attendance forecasting pipeline with deterministic trend classification and LLM interpretation (Phase 21), and a complete JWT-based authentication and role-based access control system (Phase 14).

C. Performance Summary

The following performance outcomes were established through the testing and validation activities described in Chapter VIII.

Attendance automation. The system eliminates manual attendance recording entirely for sessions where face recognition is operational. A session with 12 enrolled students would have required an estimated 5–8 minutes of manual roll-call time; with automated recognition, the professor has no attendance-related action to perform during the session. The only interaction

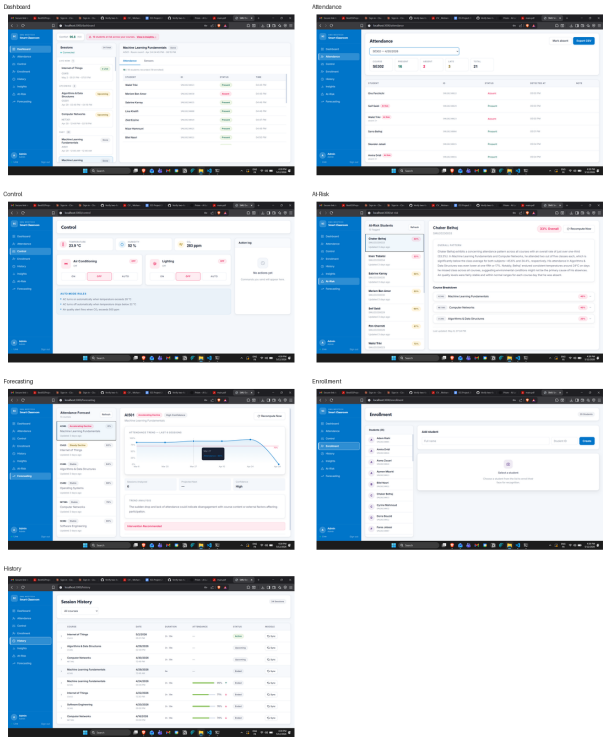

Fig. 24. Selected screenshots from the demonstration.

required is pressing the Start Session and End Session buttons, totalling under 10 seconds.

Sensor update latency. A median end-to-end latency of 85 ms from ESP32 publication to React dashboard update was achieved, well within the 5-second NFR-01 requirement. The WebSocket push architecture eliminates polling overhead entirely.

AI pipeline runtime. The at-risk pipeline processes 8 students in approximately 2–3 minutes on the Raspberry Pi 4B CPU. This runtime is acceptable given that the pipeline runs asynchronously in the background and results are progressively displayed as they become available. The forecasting pipeline processes 6 courses in approximately 1.5–2.5 minutes under the same conditions.

System resource utilization. Under normal operating conditions on the Raspberry Pi 4B (4 GB RAM), the Docker stack consumed approximately 1.8–2.2 GB of RAM across all containers (PostgreSQL, Redis, Mosquitto, FastAPI, nginx). The Ollama container with the phi3-mini model loaded consumed an additional 1.2–1.5 GB, bringing the total to approximately 3.0–3.7 GB — within the 4 GB limit but with limited headroom. During AI pipeline runs, CPU utilization reached 95–100% on all four ARM cores for the duration of the LLM inference. The system remained responsive to dashboard requests during inference due to FastAPI's async architecture.

D. Lessons Learned

Several significant technical lessons emerged during the implementation that are worth documenting for future reference.

TABLE XVII
SYSTEM OBJECTIVES VS. ACHIEVEMENT.

#	Objective	Status	Evidence
1	Reduce manual attendance overhead — automate recording, save up to 15 min per session	✓Achieved	Face recognition records attendance passively with zero professor interaction; full session of 12 students recorded automatically; manual step eliminated
2	Monitor classroom environmental conditions in real time	✓Achieved	DHT21, MQ-135, and ACP014 publish every 5s; live sparklines on dashboard; end-to-end latency median 85ms
3	Enable automated and manual environmental control	✓Achieved	Relay actuation via MQTT validated; AC auto-control on RPi and on-device ESP32; manual override via dashboard tested
4	Provide a centralized professor-facing dashboard	✓Achieved	React SPA with 9 routes; live sensor data, attendance table, relay controls, history, at-risk, forecasting in one interface
5	Generate real-time alerts for anomalous conditions	✓Achieved	temp_high, air_quality_high, device_offline alerts generated and delivered via WebSocket; LCD display confirmed
6	Synchronize attendance data with Moodle	✓Achieved	REST API sync on session end validated; Redis retry queue tested under simulated Moodle unavailability

MQTT client library migration. The initial implementation used `asyncio-mqtt` as the Python MQTT client. Following a major version release of the underlying `paho-mqtt` library (v2.0), `asyncio-mqtt` became non-functional due to a breaking API change. The project migrated to `aiomqtt`, the maintained community successor, which required updating the connection context manager syntax and reconnection handling. This experience highlighted the importance of monitoring upstream dependency changes in IoT projects where the hardware communication layer is a single point of failure.

DeepFace and TensorFlow in Docker. The initial Docker image for the backend included DeepFace and TensorFlow as dependencies. The TensorFlow installation alone pulled approximately 600 MB of dependencies and caused out-of-memory errors on development laptops with 8 GB RAM when the backend container started. The solution was to remove DeepFace from the Docker image entirely and implement a transparent stub mode for non-RPi environments. This decision reduced the Docker image size significantly, eliminated OOM crashes in development, and preserved full face recognition functionality in the Raspberry Pi production deployment where TensorFlow is installed natively.

Glassmorphism and frontend rendering. An early version of the frontend visual design used CSS `backdrop-filter:`

`blur()` for glassmorphism effects on cards and panels. During testing on the dashboard, sustained repaint cost of the blur filter caused visible frame drops on the Raspberry Pi's browser and rendering artifacts in Chromium under GPU acceleration. The glassmorphism effects were removed in Phase 18 and replaced with solid fills using the CSS token system, which resolved the rendering issues without visual regression.

Redis lock TTL and pipeline cooldown. An early implementation deleted the Redis pipeline lock immediately upon pipeline completion, so that the lock was only held during the active run. This caused the pipeline to be re-triggered on every page refresh after completion, since the lock was no longer present. The fix was deliberate: the lock TTL is set to the desired cooldown duration (600 seconds for at-risk, 1800 seconds for forecasting) and the lock is never deleted on completion. The natural TTL expiry becomes the cooldown mechanism, eliminating the need for a separate cooldown key or timestamp check.

VARCHAR vs. PostgreSQL ENUM for classification labels. The `trend_classification` column was initially defined as a PostgreSQL ENUM type. When a new classification label needed to be added during development, the `ALTER TYPE ... ADD VALUE` statement could not be executed within a transaction, making it impossible to include in an Alembic migration that would roll back cleanly on failure. The column was changed to `VARCHAR(30)`, which can be altered freely within transactions and imposes no schema migration risk as the classification vocabulary evolves.

On-demand vs. scheduled AI pipelines. An early design used APScheduler to run the at-risk pipeline on a nightly cron schedule (02:00). During testing, it became clear that professors navigating to the at-risk page during the day would see stale or empty explanations until the next scheduled run. Switching to on-demand triggering — where the pipeline fires when the page is first accessed — meant that explanations are always fresh relative to the most recent session data, regardless of when the professor accesses the page.

E. Known Issues and Workarounds

Table XVIII summarizes the known issues identified during testing and the workarounds applied or planned.

TABLE XVIII
KNOWN ISSUES AND WORKAROUNDS.

Issue	Workaround
Face recognition accuracy degrades under face mask occlusion (∼42% TPR)	Manual attendance fallback; QR-code self-check identified as future work
Face recognition accuracy drops under reduced lighting (∼78% TPR at 200 lux)	IR illuminator addition identified as future hardware upgrade
MQ-135 not calibrated against certified CO_2 reference	Treated as comparative proxy; threshold tuned empirically; replacement with NDIR sensor planned
phi3-mini inference 10–28 s per call on RPi CPU	Asynchronous background pipelines; progressive frontend polling
MQTT QoS 0 may drop messages during broker restart	5-second republish cycle minimizes telemetry loss; relay commands reissuable via dashboard
HTTP/WS deployment without TLS	Acceptable on closed LAN; nginx SSL termination documented as priority upgrade
Glassmorphism caused Chromium artifacts on RPi browser	Removed in Phase 18; replaced with solid fills via CSS token system

F. Limitations

The following limitations of the current system are acknowledged.

Single-room scope. The system is designed and validated for a single classroom (`room_id=room1`). The architecture supports multi-room extension through configuration changes, but the multi-room code paths were not tested within this project. Dashboard filtering, session management, and the alert engine would all require multi-room awareness to function correctly across multiple concurrent classrooms.

MQ-135 calibration. The air quality readings from the MQ-135 sensor are comparative and have not been validated against a certified reference instrument. Absolute PPM values should not be reported as certified measurements. The sensor is suitable for threshold-based alerting but not for regulatory compliance reporting.

Face recognition under adverse conditions. Recognition accuracy degrades substantially under reduced lighting (∼78% TPR) and near-completely under face mask occlusion (∼42% TPR). In environments where face masks are worn, the system would need to fall back to manual attendance or an alternative identification method such as QR-code check-in.

phi3-mini inference speed on RPi CPU. LLM inference on the Raspberry Pi 4B ARM Cortex-A72 CPU takes 10–28 seconds per call due to the absence of GPU acceleration. For datasets larger than the 35-student demo (e.g. 200+ enrolled students with 30+ at-risk), the pipeline runtime would become impractical without hardware upgrade (Raspberry Pi 5, or a host with a CUDA-capable GPU) or model replacement with a faster quantized variant via llama.cpp.

No HTTPS/WSS. The current deployment does not implement TLS. Tokens transmitted over HTTP on the LAN are not encrypted in transit. While acceptable for a closed university intranet, this represents a security gap that must be addressed before any public or multi-campus deployment.

MQTT QoS 0. All MQTT messages use fire-and-forget delivery with no acknowledgement. During a Mosquitto broker restart, messages published by the ESP32 between the broker going offline and reconnection are lost permanently. For sensor telemetry published every 5 seconds, this loss is acceptable; for relay command delivery, a missed command could leave the AC or lighting in an incorrect state until the next explicit command is issued. "'latex

X. CONCLUSION & FUTURE WORK

A. Summary of Contributions

This report has presented the design, implementation, and validation of the Smart Campus Classroom Management System — an IoT-based platform developed for Mediterranean Institute of Technology (SMU) that automates student attendance tracking,

monitors and regulates classroom environmental conditions in real time, and provides professors and administrators with a centralized, data-driven dashboard for session management and academic analytics.

The system was delivered across twenty-two iterative development phases within a single academic semester by a five-person student team. All six specific objectives defined at the outset of the project were fully achieved, and three additional capabilities beyond the original proposal scope were implemented: an LLM-powered at-risk student detection pipeline, an attendance trend forecasting pipeline with deterministic classification, and a complete JWT-based authentication and role-based access control system.

The project makes the following technical contributions:

A fully on-premises IoT and edge AI stack. The system demonstrates that a complete classroom management platform — including real-time sensor ingestion, face recognition attendance, relay-based environmental control, a full-featured web dashboard, and local LLM inference — can be deployed on a single Raspberry Pi 4B without any dependency on cloud services, external APIs, or internet connectivity. This architecture preserves student data sovereignty, eliminates recurring operational costs, and remains functional in network-constrained environments.

A practical hybrid deterministic-plus-LLM analytics design. The forecasting pipeline establishes a replicable pattern for using LLMs responsibly in institutional analytics: structured classification decisions are made by deterministic algorithms whose outputs are always valid and reproducible, while the LLM is constrained to prose generation tasks where natural language flexibility adds value and structured correctness is not required. This separation prevents hallucinated labels from corrupting downstream logic while still delivering interpretable, human-readable outputs.

A production-quality containerized deployment. The entire stack is deployable from a single `docker compose up -d` command on any x86 or ARM host, with Alembic auto-migration, a seeded demo dataset, a mock sensor mode, and a frontend demo fallback — making the system immediately evaluable and demonstrable without physical hardware.

B. Future Work

The following directions for future development have been identified based on the limitations documented in Chapter IX, feedback from the system demonstration, and the broader vision of a campus-wide smart infrastructure.

Multi-room support. Extending the system to cover all 30+ classrooms at Mediterranean Institute of Technology (SMU) requires making `room_id` a first-class dimension across the dashboard, alert engine, session management, and AI pipelines. The software architecture already uses `room_id` as a namespace in MQTT topics and Redis keys; the primary work is in the frontend routing and UI to allow professors and administrators to switch between rooms and compare cross-room analytics.

React Native mobile application. A mobile companion application for professors would allow session management, live sensor monitoring, and alert receipt directly on a smartphone without requiring a laptop or desktop browser. The existing FastAPI REST and WebSocket APIs are fully compatible with a React Native client with no backend changes required.

IR illuminator for low-light face recognition. The substantial drop in face recognition accuracy under reduced lighting ($\sim$78% TPR at 200 lux) can be addressed by adding an infrared illuminator module to the camera assembly. The Raspberry Pi Camera Module v2 supports near-infrared wavelengths natively, and many IR illuminators are available in the 850nm range compatible with the camera sensor. This hardware addition would significantly improve recognition reliability in evening or artificially lit classroom environments.

Email and SMS alert notifications. The current alert system delivers notifications to the dashboard and the classroom LCD. Adding outbound email (via SMTP) or SMS (via a provider such as Twilio) would allow professors to receive alerts on their personal devices when they are not actively monitoring the dashboard — for instance, a `device_offline` alert sent to the professor's phone before the start of a scheduled session.

QR-code student self-enrollment. Face image enrollment currently requires an administrator to upload images through the dashboard. A self-service QR-code enrollment flow — where students scan a session-specific QR code, are prompted to take a selfie on their phone, and the image is submitted to the enrollment API — would significantly reduce the administrative overhead of building the face encoding database at scale.

CO_2 sensor calibration and replacement. The MQ-135 sensor provides a comparative air quality proxy but cannot produce certified absolute CO_2 readings. Replacing it with a calibrated NDIR sensor (such as the Sensirion SCD30 or MH-Z19) would enable accurate CO_2 monitoring in PPM, supporting regulatory compliance reporting and more precise correlation analysis between air quality and student performance.

Offline SQLite fallback. In scenarios where the Raspberry Pi loses power or the Docker stack fails to start, the system is currently unavailable. An offline fallback mode — where the ESP32 stores sensor readings locally in flash memory and the RPi switches to a lightweight SQLite database for session recording — would improve availability for critical attendance tracking scenarios.

Daily at-risk email digest. Rather than requiring professors to navigate to the at-risk page to trigger the pipeline, a scheduled daily digest could be sent to each professor summarizing the at-risk status of students in their courses, with links to the full dashboard view. This proactive delivery model would improve the likelihood of early academic intervention.

Quantized GGUF model via llama.cpp. The phi3-mini model served via Ollama on the Raspberry Pi CPU achieves 10–28 seconds per inference call. The llama.cpp runtime, which supports GGUF-format quantized models optimized for ARM NEON intrinsics, can achieve 2–4$\times$ faster inference on the same hardware without GPU acceleration. Migrating the Ollama deployment to a llama.cpp-backed server would reduce pipeline

runtimes substantially, making the AI features more responsive for larger student cohorts.

HTTPS and WSS for public deployment. As described in Chapter VII, adding SSL/TLS termination at nginx is a straightforward configuration change. Completing this upgrade would make the system suitable for deployment on a public-facing hostname or across multiple campus buildings connected by a wider network, rather than being limited to a single LAN segment.

C. Closing Remarks

The Smart Campus Classroom Management System demonstrates that the convergence of affordable IoT hardware, containerized software deployment, open-source machine learning libraries, and locally-hosted language model inference makes it possible for a small student team to build a production-quality smart classroom platform within the constraints of a single academic semester. The system addresses real operational pain points at Mediterranean Institute of Technology (SMU) and provides a technically rigorous, extensible foundation for further development toward a campus-wide smart infrastructure.

The architectural decisions made throughout the project — edge-first deployment, event-driven asynchronous backend, deterministic classification with LLM prose generation, and a CSS token-driven frontend — reflect engineering principles applicable beyond the specific domain of classroom management and are relevant to any IoT system that must operate reliably at the edge, handle real-time data streams, and deliver AI-powered insights without sacrificing data privacy or operational independence.

REFERENCES

[1] A. Al-Fuqaha, M. Guizani, M. Mohammadi, M. Aledhari, and M. Ayyash, "Internet of things: A survey on enabling technologies, protocols, and applications," *IEEE Communications Surveys & Tutorials*, vol. 17, no. 4, pp. 2347–2376, 2015.

[2] S. Amendola, R. Lodato, S. Manzari, C. Occhiuzzi, and G. Marrocco, "RFID technology for IoT-based personal healthcare in smart spaces," *IEEE Internet of Things Journal*, vol. 1, no. 2, pp. 144–152, Apr. 2014.

[3] U. Satish, M. J. Mendell, K. Shekhar, T. Hotchi, D. Sullivan, S. Streufert, and W. J. Fisk, "Is CO_2 an indoor pollutant? direct effects of low-to-moderate CO_2 concentrations on human decision-making performance," *Environmental Health Perspectives*, vol. 120, no. 12, pp. 1671–1677, Dec. 2012.

[4] M. E. Fang, L. R. Hsu, and C. T. Yang, "A smart classroom framework based on IoT and learning analytics," in *Proc. IEEE International Conference on Applied System Innovation (ICASI)*, Sapporo, Japan, 2017, pp. 1289–1292.

[5] R. Jain and S. Agarwal, "RFID-based automated attendance system for educational institutions," *International Journal of Engineering Research and Technology*, vol. 6, no. 3, pp. 112–117, Mar. 2017.

[6] K. P. Teo and M. H. Lim, "QR code based attendance system with time-limited tokens for classroom management," in *Proc. IEEE 10th International Conference on Information Technology and Electrical Engineering (ICITEE)*, Kuta, Indonesia, 2018, pp. 430–435.

[7] H. Zhang, F. Zhang, and W. Liu, "Automatic student attendance system using face recognition in classroom environments," *Journal of Ambient Intelligence and Humanized Computing*, vol. 11, no. 4, pp. 1453–1463, Apr. 2020.

[8] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A unified embedding for face recognition and clustering," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Boston, MA, USA, 2015, pp. 815–823.

[9] S. Serengil and A. Ozpinar, "HyperExtended LightFace: A facial attribute analysis framework," in *Proc. IEEE International Conference on Engineering and Emerging Technologies (ICEET)*, Istanbul, Turkey, 2021.

[10] T. Nguyen and A. Pham, "Performance benchmarking of deep face recognition models on ARM-based single-board computers," *IEEE Embedded Systems Letters*, vol. 13, no. 2, pp. 74–77, Jun. 2021.

[11] European Data Protection Board, "Guidelines 3/2019 on processing of personal data through video devices," European Data Protection Board, Tech. Rep. ver. 2.0, Jan. 2020, https://edpb.europa.eu.

[12] A. Stanford-Clark and H. L. Truong, "MQTT for sensor networks (MQTT-SN) protocol specification," IBM and Eurotech, Tech. Rep. ver. 1.2, Nov. 2013.

[13] OASIS Standard, "MQTT version 5.0," OASIS, Tech. Rep., Mar. 2019, https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html.

[14] L. Tan and N. Wang, "Future internet: The internet of things," in *Proc. IEEE 3rd International Conference on Advanced Computer Theory and Engineering (ICACTE)*, Chengdu, China, 2010, pp. V5–376–V5–380.

[15] T. Hagendorff, "The ethics of AI ethics: An evaluation of guidelines," *Minds and Machines*, vol. 30, no. 1, pp. 99–120, Mar. 2020.

[16] Microsoft Research, "Phi-3 technical report: A highly capable language model locally on your phone," Microsoft Research, Tech. Rep. arXiv:2404.14219, Apr. 2024.

[17] Ollama, "Ollama: Get up and running with large language models locally," GitHub repository. https://github.com/ollama/ollama, 2024.

[18] Moodle HQ, "Moodle web services API documentation," Moodle Developer Resources. https://docs.moodle.org/dev/Web_services, 2024.

[19] B. Alturki and W. Guthier, "Enhancing learning using internet of things in higher education," *International Journal of Advanced Computer Science and Applications*, vol. 10, no. 7, pp. 361–367, 2019.

[20] M. Rahman, A. Islam, and S. Roy, "IoT-based smart classroom attendance system using RFID," in *Proc. IEEE International Conference on Robotics, Electrical and Signal Processing Techniques (ICREST)*, Dhaka, Bangladesh, 2019, pp. 216–221.

[21] R. Singh, P. Sharma, and A. Verma, "Mobile QR-based attendance management system with cloud synchronization," *International Journal of Computer Applications*, vol. 181, no. 32, pp. 14–19, May 2019.

[22] J. Won and S. Park, "Real-time face recognition attendance system with Moodle LMS integration," in *Proc. IEEE International Conference on Big Data and Smart Computing (BigComp)*, Kyoto, Japan, 2020, pp. 398–401.

[23] F. Al-Turjman, H. Zahmatkesh, and R. Shahroze, "An overview of security and privacy in smart cities' IoT communications," *Transactions on Emerging Telecommunications Technologies*, vol. 30, no. 10, p. e3677, 2019.

APPENDIX A
FULL API ENDPOINT REFERENCE

TABLE XIX
REST API ENDPOINT REFERENCE (APPENDIX A, PART 1).

M	Endpoint	Auth	Role	Description
POST	/api/auth/login	None	Any	Issue JWT on valid credentials
GET	/api/sensors/latest	JWT	Any	Latest sensor readings from Redis
GET	/api/sensors/history	JWT	Any	Historical readings (filter: room_id, type, from, to)
GET	/api/sessions/{id}/sensors/latest	JWT	Any	Per-type sensor readings within session window
GET	/api/sessions/{id}/sensors/summary	JWT	Any	Avg/min/max per sensor type (session must be ended)
POST	/api/sessions/start	JWT	Prof.	Start new session, launch recognition loop
POST	/api/sessions/{id}/end	JWT	Prof.	End session, bulk-insert absent, trigger Moodle sync
GET	/api/sessions	JWT	Any	Filtered session list
GET	/api/sessions/{id}	JWT	Any	Single session detail
POST	/api/control/ac	JWT	Prof.	Set AC relay (on/off/auto)
POST	/api/control/lighting	JWT	Prof.	Set lighting relay (on/off/auto)
GET	/api/control/status/{room_id}	JWT	Any	Relay states + live sensors + device online
GET	/api/alerts	JWT	Any	List alerts (filter: room_id, acknowledged, limit)
PATCH	/api/alerts/{id}/acknowledge	JWT	Any	Mark alert acknowledged
GET	/api/alerts/unread-count/{room_id}	JWT	Any	Count of unacknowledged alerts
GET	/api/courses	JWT	Any	Course list (role-filtered)
POST	/api/courses	JWT	Admin	Create course
GET	/api/courses/{id}	JWT	Any	Single course detail
POST	/api/courses/{id}/enroll	JWT	Admin	Enroll students in course

TABLE XX
REST API ENDPOINT REFERENCE (APPENDIX A, PART 2).

M	Endpoint	Auth	Role	Description
GET	/api/sessions/{id}/attendance	JWT	Any	Attendance records for session
PATCH	/api/attendance/{record_id}	JWT	Prof.	Manual attendance adjustment
POST	/api/sessions/{id}/mark-absent	JWT	Prof.	Bulk mark remaining students absent
GET	/api/students/{id}/attendance-history	JWT	Any	Full attendance history for student
GET	/api/students	JWT	Any	Student list
POST	/api/students	JWT	Admin	Register new student
POST	/api/students/{id}/enroll-face	JWT	Admin	Upload face images and compute embedding
GET	/api/students/{id}/courses	JWT	Any	Courses enrolled by student
GET	/api/at-risk	JWT	Any	At-risk students with LLM explanations (triggers pipeline)
GET	/api/at-risk/{student_id}	JWT	Any	Full at-risk detail for one student
POST	/api/at-risk/recompute	JWT	Admin	Force pipeline recompute (bypasses lock)
GET	/api/forecasting	JWT	Any	All course forecasts (triggers pipeline)
GET	/api/forecasting/{course_id}	JWT	Prof.	Single course forecast
POST	/api/forecasting/recompute	JWT	Admin	Force forecasting recompute
GET	/api/health	None	Any	Health check (Redis + DB connectivity)
GET	/api/moodle-test	JWT	Admin	Moodle connectivity probe
WS	/ws/classroom/{room_id}	None	Any	WebSocket live stream

APPENDIX B
MQTT TOPIC SCHEMA

All topics follow the namespace `classroom/{room_id}/...` where `room_id` defaults to `room1`. Backend subscription wildcards: `classroom/+/sensors/#` and `classroom/+/status`.

TABLE XXI
MQTT TOPIC SCHEMA (APPENDIX B).

Topic	Pub.	Sub.	Payload Schema	Interval
classroom/{room_id}/sensors/temperature	ESP32	Backend	{"value": float, "unit": "C", "ts": int}	5 s
classroom/{room_id}/sensors/humidity	ESP32	Backend	{"value": float, "unit": "%", "ts": int}	5 s
classroom/{room_id}/sensors/air_quality	ESP32	Backend	{"value": float, "unit": "ppm", "ts": int}	5 s
classroom/{room_id}/sensors/sound	ESP32	Backend	{"value": 0|1, "unit": "bool", "ts": int}	5 s
classroom/{room_id}/status	ESP32	Backend	{"online": true, "ts": int}	30 s
classroom/{room_id}/relay/ac	Backend	ESP32	{"action": "on"|"off"|"auto", "cmd_id": str}	On cmd.
classroom/{room_id}/relay/lighting	Backend	ESP32	{"action": "on"|"off"|"auto", "cmd_id": str}	On cmd.
classroom/{room_id}/alerts	Backend	ESP32	{"type": string, "value": float}	On thr.

APPENDIX C
DATABASE SCHEMA (TABLE DEFINITIONS)

TABLE XXII
POSTGRESQL SCHEMA DEFINITIONS (APPENDIX C).

Table	PK	Key Columns	Notes
professors	UUID	name, email (UNIQUE), hashed_password, role ENUM (professor—admin)	bcrypt hashed password
students	UUID	name, student_id (UNIQUE)	Institutional student ID
courses	UUID	code (UNIQUE), name, professor_id (FK → professors, nullable)	professor_id nullable for unassigned courses
course_students	(course_id, student_id)	course_id FK, student_id FK	M:M join table
sessions	UUID	course_id FK, room_id, started_at, ended_at, status ENUM (active—ended—upcoming)	display_status computed at query time
attendance_records	UUID	session_id FK, student_id FK, status ENUM (present—absent—late—excused), detected_at, adjusted_by, moodle_synced BOOL	adjusted_by stores professor UUID
face_encodings	UUID	student_id FK (UNIQUE), encoding BYTEA	128-d float32 numpy array via tobytes()
sensor_readings	UUID	room_id, sensor_type ENUM, value FLOAT, unit, recorded_at	High-volume append-only table
alerts	UUID	room_id, type ENUM, value FLOAT, message TEXT, acknowledged BOOL	Deduplicated by type per room
at_risk_explanations	UUID	student_id FK (UNIQUE), overall_attendance_rate, summary_explanation TEXT, per_course_data JSONB, generated_at, ollama_reachable BOOL	Upserted on each pipeline run
attendance_forecasts	UUID	course_id FK (UNIQUE), trend_data JSONB, expected_next_rate FLOAT, trend_classification VARCHAR(30), confidence_level VARCHAR(10), interpretation TEXT, generated_at, ollama_reachable BOOL	VARCHAR not ENUM for classification

APPENDIX D
ENVIRONMENT VARIABLE REFERENCE

See Table XIV in Section V-G for the complete environment variable reference including descriptions and default values.

APPENDIX E
HARDWARE WIRING DIAGRAM

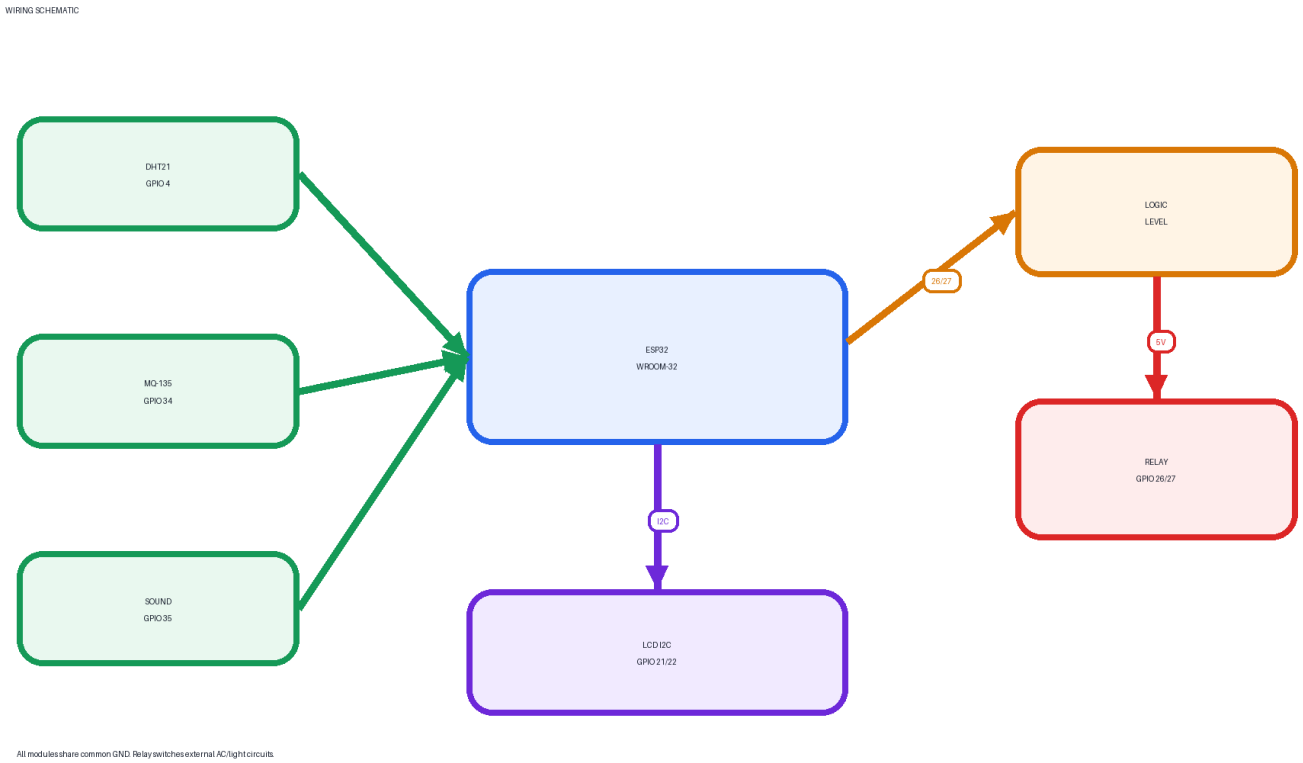

Fig. 25. Hardware wiring schematic.

Key connections summary:

TABLE XXIII
KEY WIRING CONNECTIONS.

Component	ESP32 Pin	Notes
DHT21 data	GPIO 4	10kΩ pull-up to 3.3V
MQ-135 analog	GPIO 34 (ADC)	ADC1 channel; GPIO 34 is input-only
ACP014 digital out	GPIO 35	Digital HIGH/LOW
Relay ch1 (AC)	GPIO 26 → LLC → Relay IN1	Via logic level converter
Relay ch2 (Lighting)	GPIO 27 → LLC → Relay IN2	Via logic level converter
LCD SDA	GPIO 21	I2C SDA (shared bus)
LCD SCL	GPIO 22	I2C SCL (shared bus)
LCC LV	3.3V	Low-voltage side
LCC HV	5V (VIN)	High-voltage side

TABLE XXIV
DOCKER COMPOSE SERVICES (APPENDIX F).

Service	Image	Port	Volumes	Depends On
postgres	postgres:15-alpine	5432 (int)	postgres_data:/var/lib/postgresql/data	—
redis	redis:7-alpine	6379 (int)	redis_data:/data	—
mosquitto	eclipse-mosquitto:2	1883 (int)	mosquitto_data:/mosquitto/data	—
ollama	ollama/ollama:latest	11434 (int)	ollama_data:/root/.ollama	—
backend	Custom (Dockerfile)	8000 (int)	—	postgres, redis, mosquitto, ollama
frontend	Custom (nginx)	3000 → 80	—	backend
mariadb (opt.)	mariadb:10.6	3306 (int)	mariadb_data:/var/lib/mysql	—
moodle (opt.)	bitnami/moodle:4	8080 → 8080	moodle_data:/bitnami/moodle	mariadb

APPENDIX F
DOCKER COMPOSE CONFIGURATION SUMMARY

Start commands:

```
docker compose up -d                          # core stack
MOCK_MODE=true docker compose up -d           # without ESP32
docker compose --profile moodle up -d         # with Moodle LMS
docker compose exec backend python seed.py    # seed demo data
```

APPENDIX G
NOTION TASK MANAGEMENT BOARD

Fig. 26. Notion task management board.

The Notion workspace was structured with:

- A main Kanban board with one card per development phase
- Each card containing: phase description, assigned team member(s), due date, sub-task checklist, and completion notes
- A linked database view filtered by assignee for individual workload tracking
- A shared documentation space for architecture decisions, meeting notes, and hardware setup guides